1.1 設計的關鍵是-把無形變有形.mp4

🎯 軟體設計核心理念與挑戰

> 「設計就是把無形變有形。」

- 軟體設計的最大難題在於，需求通常是模糊且無形的，自然語言表達帶有彈性和雜訊。
- 若無法清晰「看見」需求和結構，思考就像視力近似800度，腦袋無法運作，設計自然無法產生。
- 需求短小不代表簡單；即使是小需求，也可能因缺乏明確的結構和分析而導致設計失敗。

🃏 小需求實務案例：道德二撲克牌遊戲

挑戰點	內容說明
需求複雜度	文字雖少，規則多且多樣，包含牌型、出牌規則、起始玩家、勝負判定等
需求不明確	PM可能自己也不確定需求，給出模糊指示和緊迫期限
程式碼結構混亂	直覺寫出大量 if else 判斷，導致程式碼難以維護和擴充
需求變更	老闆直接加新功能，改變遊戲規則，讓程式碼難以應對變更
重構困難	面對混亂程式碼，開發者腦袋卡住，難以有效重構

🔄 重蹈覆轍的四部曲

步驟	描述
1. 從頭開始的自信	缺乏分析，急著寫程式，先寫再說
2. 陷入細節	只看程式碼細節，沒有抽象視角，失去秩序
3. 急於求成	寫完就上線，將錯就錯，忽視程式碼品質
4. 後果與重寫	需求變更導致必須重寫，浪費時間和資源

- 問題根源非個人能力不足，而是軟體本質的「混沌」特性與缺乏有效思考工具。

💡 軟體本質與語言差異

類別	特性與挑戰
需求（自然語言）	彈性大、含混雜訊、模糊不清，難以直接轉化為精確指令
程式碼（程式語言）	精準、詳細、必須明確指令，無法容忍模糊和彈性

- 這種差異使得設計和實現階段充滿細節與雜訊，缺乏抽象能力會讓設計者陷入混亂。

Thetawave.ai

🧠 抽象思考與設計的關鍵

- 抽象能力是將複雜、細節紛繁的系統「抽離」出高階結構和行為視角。
- 缺乏抽象，思維就會像近視800度，什麼都看不清楚，無法抓住「施力點」。
- 問題本質在於找不到合適的「施力點」來推進設計。

🗣️ 語言與認知推進

- **聊天模式**能刺激思考，因為語言推動問題一問一答，促進認知深化。
- **正確的問題**是設計思考的關鍵，知道怎麼問問題勝過八成高手。
- 軟體設計需要一種「語言」來推進分析與設計思路。

🖼️ UML 統一建模語言（Unified Modeling Language）

> UML 是用來推進軟體分析與設計的專業語言。

- UML協助將抽象的需求「具象化」為模型，促進有形思考。
- 透過UML建模，可以像聊天一樣不斷提出問題，推進認知與設計。
- UML的顆粒度停留在**結構（Structure）與行為（Behavior）**兩個高階視角，幫助設計師從細節中跳脫。
- 透過UML可建立**領域模型（Domain Model）**，將需求轉化為系統的清晰藍圖。

🔍 用UML推進設計思考的實務流程

階段	活動描述
接收需求	從自然語言需求出發
建立初步模型	利用UML快速繪製領域模型，將需求具象化
問問題推進認知	例如：遊戲有哪些隊列？牌的數量是多少？牌有什麼屬性？洗牌與抽牌如何實現？
逐步細化模型	不斷問問題、完善模型，實現有形思考
進階設計	根據模型分析局部問題，查表選用合適設計模式進行重構
實作程式碼	按模型結構「按圖施工」，編寫易維護且高品質的程式碼

🛠️ 設計思路四階段訓練

階段	說明
1. 建立架構思維	從宏觀角度理解系統架構與需求
2. 精通建模思路	熟練使用UML等建模工具具象化思考
3. 快速迭代梳理設計	不斷反覆推敲、修改模型，提升設計品質
4. 按圖施工	根據模型實作程式，保證程式碼品質和可維護性

Thetawave.ai

📈 設計升級的心法

- 不須過度依賴直覺，設計模式和重構技巧可以透過分析和查表快速決定。
- 抽象思維提供「施力點」，讓設計者思考更有效率且更精確。
- 設計能力是後天養成，持續投資在有形思考和模型建構上，才能獲得高效流暢的軟體設計。

軟體設計的核心在於，從混沌的自然語言需求中抽離出結構與行為的清晰模型，並以此模型為基礎，推動設計思路的發展與重構決策，最終實現高品質且可維護的程式碼。這種抽象與具象的雙重轉換使設計者能找到施力點，打破思維混亂，實現軟體設計的行雲流水。

1.2 UML 不是拿來寫文件用的.mp4

UML 非文件工具的核心觀念 🎯

> **UML（統一建模語言，Unified Modeling Language）不是用來寫文件或產生附圖的工具，而是一種幫助思考與推進設計思路的建模語言。**

- 常見迷思：許多人誤以為 UML 僅是繪圖工具，並將其當成附在文件上的圖示，這是錯誤的認知。
- UML 的真正價值在於協助設計師和團隊**快速且準確地理解需求，進而分析與設計系統架構**。

需求到程式語言的思考流程 🌐

階段	說明
自然語言需求	以自然語言描述的需求，是最初的輸入資訊。
UML 建模語言	將需求轉換成有結構、可分析的模型，幫助思考與溝通。
程式語言	根據模型撰寫程式碼，精確指令機器執行。

- UML 是連結「自然語言需求」與「程式語言」的橋樑，讓設計過程更有條理和效率。

敏捷開發與文件迷思 🔁

- 傳統常用大量文件與附圖與客戶溝通，因溝通管道不暢，文件量大、更新慢，導致文件容易過時且缺乏彈性。
- **敏捷宣言強調「個人與互動勝過流程與工具」**，鼓勵減少文件負擔，強調團隊面對面溝通。
- 然而，這並不代表 UML 過時，而是要**將 UML 視為思考與溝通的工具，而非死板的文件產物**。

常見疑惑與核心問題 💡

Thetawave.ai

疑惑	說明與解答
UML 是否因敏捷而過時？	不過時。只要需要思考、分析和複雜設計，就需要 UML 或類似建模語言來幫助。
系統分析師角色消失，還要用 UML？	角色可能變少，但抽象思考與設計能力依然重要，UML 是推進設計思路的工具。
UML 如何維護？	UML 是工具，不是文件。維護重點在於持續思考和更新設計，而非維護一堆靜態文件。
UML 畫得不清楚，怎麼辦？	重點是掌握思考流程與模型清晰度，並非追求完美圖形。只要能有效溝通與推理，足以支撐設計。

設計思考的重要性：軟體抽象能力 🧠

> **軟體抽象能力是軟體架構師、CTO 等技術決策者必備的核心能力。**

- 不只是寫程式，更重要的是如何從需求拆解、多層次抽象思考。
- 從需求 → 系統架構 → 程式架構 → 實際程式碼，這條路徑要在動手寫程式前就有清晰藍圖。
- 沒有圖形與模型的輔助，設計複雜系統如同盲目下棋，缺乏形狀感就無法成為高手。
- UML 是第二大腦，幫助把無形的思考抽象化為可視化的結構與行為。

UML 主要模型種類與本課使用重點 📐

UML 模型類型	說明	本課使用重點
結構型模型	聚焦系統中「點與線」的結構與關係，例如類別與關聯。	主要用：類別圖 (Class Diagram)
行為型模型	聚焦系統「行為」及其隨時間的變化，例如動作流程與狀態。	主要用：循序圖 (Sequence Diagram)、狀態圖 (State Diagram，第三章教授)

- 本課重點在於掌握**類別圖**、**循序圖**兩種模型，足以設計複雜系統與框架。
- 狀態圖用於更複雜行為分析，會在後續章節學習。

UML 使用的關鍵：思路推進而非工具本身 ⚙️

- 工具只是載體，**重點是武功（設計思考與建模能力）怎麼打**。
- 熟練 UML 思考流程，能輕鬆設計出複雜系統架構。
- 不追求過度複雜的 UML 規範，彈性運用即可達成目標。

具體思考演練問題 🔍

- 如果什麼都不畫，你能直接設計出複雜框架的完美架構嗎？
- 你能在不寫程式的情況下，分析、設計並預測潛在問題，知道何處應用哪些設計模式嗎？
- 沒有圖形和模型，面對大量需求文字，思考如何起步？

答案是：**不可能。思考必須藉由模型來具象化，UML 正是這樣的工具。**

Thetawave.ai

以圍棋作為比喻 🎲

- 學習圍棋必須先學「有形狀的棋形」，了解好形、壞形。
- 類比於軟體設計，必須先掌握「建模語言」以理解程式碼的抽象形狀。
- UML 就像視覺化的棋盤形狀，幫助掌握整體架構與細節。

本課程 UML 使用說明 📘

- 不會大量依賴 UML 規範細節，甚至會超越規範彈性運用。
- 只需熟悉類別圖與循序圖，能快速分析與設計。
- 狀態圖視需求在後續章節深入學習。
- 目標是用 UML 促進設計思路，而非產生文件。

以上內容反映 UML 在軟體分析與設計中的核心地位，並突顯其真正價值是推動設計思路與抽象思考，而非僅僅作為文件繪圖工具。透過掌握結構型與行為型的核心模型，能有效應對複雜系統設計，提升軟體架構品質。

1.3 行雲流水的OOA _OOD _OOP.mp4

💡 迭代式物件導向分析與設計框架

> **行雲流水的迭代事務件導向設計** 是本課程的核心方法論，貫穿整個軟體設計模式的學習旅程。
> 透過迭代，從60%設計完整度開始，不斷優化設計，避免一次到位的強迫症，讓設計越來越好。

主要學習目標

- **輕鬆寫出好維護且具擴充性的程式碼**
- **擁有職場上的核心競爭力**
- **能參與複雜系統設計決策，並且表達具說服力**
- **核心目標：給定任何需求，能透過系統化思路，快速且精準產出高品質程式碼**

> 高品質程式碼的三大要素：
>
> **準確交付正確功能**
>
> **剛剛好的擴充性**（避免過早優化）
>
> **易讀且易維護**

🧭 三大迭代關注點（OA、OD、OP）

Thetawave.ai

階段	名稱	英文全名	主要任務描述	關注點
1	物件導向分析 (OA)	Object Oriented Analysis	從需求中萃取結構化知識，理解需求，建立領域模型	做對的事：理解需求，萃取領域知識
2	物件導向設計 (OD)	Object Oriented Design	針對分析結果，分析設計難題，套用設計模式，產生設計藍圖	把事做對且做好：優化架構
3	物件導向程式設計 (OP)	Object Oriented Programming	按設計藍圖實作程式碼，快速準確交付新版本程式	快狠準寫程式碼

三大關注點詳細說明

1. 物件導向分析（OA）

- **目的：** 從需求中萃取具體的領域知識與規則，建立領域模型
- **重點：**
  - **理解問題領域（Problem Domain）**，即需求背後的專業領域
  - 使用 **UML（統一建模語言）** 幫助「戴上眼鏡」，看懂需求與問題
  - 建立「領域模型」，這是表達領域中重要知識、概念與規則的模型，比喻成玩具車模型，雖然不能動，但從模型可看出車子有四個輪子
  - 同時萃取非功能性需求，如擴充性需求、變動點（用「約束」或Forces表示）
- **注意：** 不可同時做架構設計，避免思緒混亂與設計失焦

2. 物件導向設計（OD）

- **目的：** 分析領域分析階段的約束與設計難題，套用適合的設計模式（Design Patterns）
- **重點：**
  - 根據約束，精準選擇設計模式，提高系統的維護性與剛剛好的擴充性
  - 產生具備高品質的設計藍圖
- **注意：** 不仰賴直覺，須有條理、有依據地設計

3. 物件導向程式設計（OP）

- **目的：** 根據設計藍圖實作程式碼
- **重點：**
  - 按圖施工，快速且準確交付程式碼
  - 實作時若發現設計不完善，允許回到 OD 迭代調整設計
  - 若遇需求不明確或錯誤，回到 OA 重新建模與理解

🔄 迭代設計流程與心態

- **迭代，非一次到位**
  - 常見誤解是要一次完美設計，實務上非常困難且不必要
  - 允許多次往返 OA、OD、OP 階段，逐步完善設計與實作
- **迭代心法**
  - 寫程式（OP）發現問題，退回設計（OD）修正
  - 設計不清楚，退回分析（OA）深化領域模型
  - 保持思考推進，避免陷入細節或完美主義
  - 讓思路始終有施力點，清楚下一步行動

Thetawave.ai

📊 迭代階段建議完成度

階段	建議完成度	說明
OA	約 80%	需求讀過一次，領域模型大致完成即可進入設計
OD	約 60%	不追求完美，先完成初步設計藍圖開始實作
OP	持續進行	按設計寫Code，遇問題回到OD或OA迭代

🧠 思考流程核心原則

原則	說明
有施力點	每個思考步驟都基於明確基準點，避免無根據的直覺判斷
關注點分離	將分析、設計、實作三個步驟分開思考，一次聚焦一個關注點，減輕大腦負擔
有條理且有據	所有設計決策必須有明確依據，口頭表達時具說服力
結合需求與領域	需求是起點，但必須理解背後的問題領域，才能正確提煉知識並設計系統

重要名詞定義

名詞	定義
需求（Requirement）	包含冗長規格文件、User Story、口頭指示等，皆屬於軟體開發的需求輸入
問題領域（Problem Domain）	業務方所在的專業領域，軟體需解決的實際問題範圍
領域模型（Domain Model）	表達問題領域重要知識和規則的模型，是物件導向分析的核心成果
約束（Forces）	需求中可能引發變動的非功能性條件，例如擴充性、維護性需求，設計時需特別考慮
設計模式（Design Pattern）	經過驗證可解決特定設計問題的通用方案，讓系統具備良好維護性和擴充性

例子說明

- **玩具車的領域模型**
  如玩具車模型雖無法真正動，但可清楚看到「車有四輪」這一不變的領域知識。
- **電商系統領域模型**
  包含訂單(Order)、訂單項目(Order Item)、產品(Product)、購物車(Shopping Cart)、購物車項目(Shopping Cart Item)、使用者(User)等概念。
  透過模型清楚表達電商領域的核心元素與關係。

迭代流程比喻

> 迭代設計如同「行雲流水」的水流，流動中不斷調整方向與力度，不追求一次到位的完美，卻能持續推進，達到順暢且高效的設計成果。

Thetawave.ai

物件導向分析、設計與程式設計的三個階段相互關聯，彼此間的迭代能使需求理解、架構設計與程式實作逐步完善，形成一個高品質且具備擴充性、易維護的軟體系統。這種方法論不僅讓設計思路清晰且有力，也實際提升軟體交付的速度與品質，並讓工程師在面對各種挑戰時能保持自信與說服力。

1.4 上這門課一定要有空杯心態.mp4

🎯 學習心態與課程定位

> **空杯心態**是本課程學習的最重要基石。即使具備豐富實戰經驗或已經讀過相關書籍，仍需拋開舊有思維，才能真正吸收新知。

- 學員背景多元，包含技術決策者（如 CTO、架構師）及初階工程師。
- 多數學員（約 80%）已有自行閱讀過設計模式相關書籍的經驗。
- 學員多數在實戰中累積了大量「經驗法則」（直覺式決策），此課程旨在修正、精煉這些直覺，建立**有條有理、有憑有據的建模思維**。

📚 主要學習策略

策略名稱	重點說明	目的
空杯心態	放下既有的舊思維，不用直覺判斷封裝或介面設計，避免錯誤決策。	確保新思路能被有效吸收並內化。
全心專注教學內容	不在課程中展現已知知識，專注於磨練老師教授的思維方法。	避免混淆，形成穩固且通用的設計思考。
動手實踐	隨時使用 YAML Editor 停止影片練習，落實操作，不只看影片。	透過大量練習深化理解，提升應用能力。
反覆練習	利用「練舞影片」形式手把手練習，並完成所有道館作業挑戰，使用課程思維方式做題。	內化設計模式思維，強化職場實戰能力。

💡 常見誤區與澄清

誤區	正確認知
以「直覺」判斷哪些類別該封裝、哪些介面該開放。	直覺往往缺乏證據與系統性，設計決策應基於明確理論與實證。
依照過去習慣思考設計，忽略學習新概念。	需放下舊有思維（倒掉杯子裡的水），專注吸收新方法，才能提升設計品質。
僅看影片而不動手練習。	只看影片容易淺嘗輒止，動手練習才能真正內化思考流程與技巧。
不用課程教的方式做作業，仍沿用過去做法。	用課程方法做題，透過 Code Review 才能獲得最大學習效果。

🎥 課程學習工具與影片形式

- **YAML Editor**
  學員需要隨時打開此工具，方便即時練習。
  應避免「懶惰」不動手，運用工具能大幅提升學習效率。
- **練舞影片（Practice Videos）**
  不再講解理論，而是手把手帶領完成題目。
  學員必須停影片動手操作，確保思維轉化為實際能力。

- **道館作業挑戰**
  通過完成挑戰可獲得勳章，激勵學習。
  必須用課程教的方式完成，避免回到舊路徑。

Thetawave.ai

🔑 核心概念總結

概念	定義與說明
空杯心態（Empty Cup Mindset）	拋開舊有經驗與直覺，心無旁騖地接受新知識，有如空杯般才能裝滿新水。
設計決策（Design Decision）	基於證據和系統性原理的設計選擇，而非憑直覺判斷，包含封裝類別與介面設計。
內化（Internalization）	透過實際操作與反覆練習，使新學的思路成為自身工作流程中的自然部分。
學習機動性（Learning Agility）	主動練習、主動思考，決定了學習成效的高低。

🧠 心態比喻

將學習比喻成「杯子裝水」：

- 學員的「杯子」裡已裝滿經驗的「水」。
- 若不倒掉舊水（舊思維），新水（新知識）無法裝下。
- 必須先倒空杯子，才能用課程的知識「重新裝滿」，這樣才能形成更有效率且通用的設計思考。

💼 課程價值與學習意義

- 學費是辛苦錢，投入學習即是對職涯的投資。
- 掌握底層技法，能在職場中提升影響力，爭取頂標年薪。
- 面對未來生成式人工智慧（Generative AI）等挑戰，深厚的設計能力是競爭優勢。
- 課程的學習方式與內容，將使學習效果倍增，打造穩健的專業核心。

本課程強調心態的調整與系統性思考，結合實作練習，能使軟體設計思維達到行雲流水般的境界，成為職場中真正萬用且有效的工具。

1.6 軟體抽象之一_點的萃取.mp4

🟢 物件導向分析（OOA）中的「點」萃取

> **物件導向分析（Object-Oriented Analysis, OOA）**的核心在於從需求中抽取出系統中的重要「點」與「線」，其中「點」指的是物件類別與屬性，是結構模型中的基礎元素。

---

1. 物件導向分析的目標與流程

- **目標**：從需求中剝除雜訊，萃取出只涵蓋重點知識或要素的類別圖（Class Diagram）。
- **流程**：
  1. **先萃取點（物件類別與屬性）**，再萃取線（類別間關係）。
  2. 聚焦於「點」的萃取，逐步建立精準細膩的分析思路。

---

2. 「點」的定義：物件、實體與類別

Thetawave.ai

名稱	定義	範例
實體 Entity	任何可實際存在的物體或資料體（不限定於現實世界，資料庫中的資料也算）	象棋棋子、棋盤、資料庫中的使用者資料
物件 Object	已存在且可辨識的實體，有唯一身份	目前遊戲中的紅色帥棋子
類別 Class	抽象的分類名詞，不代表任何已存在且唯一的物件，是物件的集合	象棋棋子（抽象類別，不指特定棋子）

> **比喻**：就像生物學家先觀察各種動物（物件），再根據共通特徵分類成不同物種（類別）。

---

3. 物件先於類別的分析思維

- **錯誤觀念**：許多人誤以為從類別開始分析（受程式語言如 Java、C# 影響）。
- **正確觀念**：必須先從「物件」開始，辨識出實際存在的實體，再根據物件的屬性與行為交集做分類形成類別。
- **實例**：
  - 象棋中先找出32個棋子物件與棋盤物件，再分類為棋子類別與棋盤類別。

---

4. 類別的定義與物件分類標準

- **類別 = 物件的交集**（屬性與行為的交集）
- **分類標準**：
  1. 找出一群物件
  2. 找出它們共通的屬性與行為
  3. 用一個類別名稱表示這些共通特徵
- **範例**：
  兩隻狗（法鬥、小藏獒）
  - 共通屬性：種類、名字、體重、身高
  - 共同行為：吃喝拉撒、汪汪叫
  - 類別：小狗

---

5. 屬性（Attribute）的定義與萃取

> **屬性**是用來描述某個物件或類別的特徵值（Value），本身不具備身份，僅為描述用途。

- **屬性例子**：
  - 棋子：顏色（黑色、紅色）
  - 棋盤：縱線（9條）、橫線（10條）
- **屬性型別**與**基數（Multiplicity）**：
  - 型別：屬性資料的種類（如整數、字串）
  - 基數：該屬性可有幾份（預設為1，亦可為多份如陣列）
- **基數範例**：
  - 顏色（以整數表示，0=黑色，1=紅色）
  - RGB三原色可用基數3表示三個整數值
  - 密碼長度限制可用字元型別加基數表示（如16-23字元）

Thetawave.ai

6.「點」萃取 SOP（標準作業流程）

步驟序號	問題	判斷及行動
1	詞彙是否具備特定意義？	若沒有，忽略；有特定意義則繼續判斷
2	該詞彙是否代表可實際存在的實體？	是 → 物件，否 → 屬性
3	物件是否代表唯一且已存在的實體？	是 → 物件，否 → 類別
4	將物件分類成類別：找出物件的屬性與行為交集，命名類別	建立類別圖中的點（類別名稱與屬性）

---

7. 類別圖中的「點」表示方式

- 類別名稱置於方框頂部
- 屬性列於類別名稱下方，格式為：

  屬性名稱 : 型別 [基數]

  例如：

  或帶基數：
- 屬性可附註說明（如顏色0代表黑色，1代表紅色）

---

8. 物件導向分析與程式實作的關係

分析階段	重點	實作階段	重點
物件導向分析（OOA）	從需求中分析物件與類別，抽取知識結構	物件導向程式設計（OOP）	根據類別寫程式碼，建立物件
先有物件，再分類出類別	以需求為核心，不考慮程式語言細節	先有類別，再透過類別產生物件	以程式碼為中心

> 物件導向分析的成果有助於程式設計階段的結構清晰與程式簡潔。

---

9. 專有名詞總整理

名詞	英文	定義
實體	Entity	可實際存在的物體或資料體（現實或資料庫皆可）
物件	Object	已存在且具有唯一身份的實體
類別	Class	抽象分類名詞，代表物件的屬性與行為交集
屬性	Attribute	用來描述實體特徵的值，不具身份
基數	Multiplicity	屬性或關係的數量限制
實體化	Instantiation	由類別產生具體物件的過程

---

10. 練習與應用示例

- **象棋例子**：
  - 物件：32個棋子、1個棋盤
  - 類別：棋子類別、棋盤類別

Thetawave.ai

    - 屬性：棋子顏色（整數）、棋盤縱線（整數）、橫線（整數）
- **Discord例子**：
    - 物件：頻道、貼文、成員
    - 類別：頻道類別、貼文類別、成員類別

---

11. 萃取「點」的思維練習提示

- 針對需求中「具特定意義」的詞彙進行辨識。
- 先想像或觀察物件（已存在的實體）。
- 根據物件的共通屬性與行為分類為類別。
- 屬性用於描述類別的特徵，非實體存在。
- 切勿混淆物件與類別，或將類別當成程式中的藍圖直接使用。

物件導向分析的「點」萃取透過先辨識實際存在的物件，再根據屬性與行為交集分類出類別，形成精確的系統知識結構，這種由上而下的結構化分析為後續設計與實作奠定穩固基礎。

1.7 練武 - 點的萃取.mp4

📝 「點」的萃取分類練習 SOP（標準作業程序）

> 「點」的萃取指的是將詞彙分類成三種基本類型：**物件**、**類別**（Class，分類名詞）或**屬性**，透過一套有條理的判斷流程（SOP）來決定每個詞彙的正確分類。

🧩 分類核心判斷流程（SOP）

判斷問題序列	內容說明	判斷標準	結果示例
問題 1	詞彙是否代表「可實際存在的實體」？	是→繼續問題 2；否→屬性	電池：是；電量：否
問題 2	詞彙是否為「存在且唯一的實體」？	是→物件；否→類別	小白（鄰居的小狗）：是；人類：否

🌟 重要定義與概念

- **物件（Object）**：存在且唯一的實體。例如：「太陽」、「水韭潘」、「今天早上做的第二件事」、「台灣」、「軟體設計模式精通之旅（指定的課程）」。
- **類別（Class）**：抽象分類名詞，代表一組具有共同特徵的實體，但並非單一獨特個體。例如：「人類」、「水球軟體學院的課程」、「水韭潘的手機」。
- **屬性（Attribute）**：描述物件或類別的特徵或性質，通常是不可實體化的特徵值。例如：「電池的電量」、「顏值」、「曖昧的程度」、「事情的優先序」、「店家的地址」。

🧠 判斷細節說明與易混淆點

Thetawave.ai

1. 可實際存在的實體

- 實體指的是「在現實中可以被辨識存在」的東西。
- 例如，「電池」是有形的實體；「電量」是描述電池的特徵，非具體實體。
- 在類別圖中，屬性本身被視為實體（可見、可辨識元素），但在語意分類時仍屬抽象分類，故判定為類別。

2. 存在且唯一

- 唯一性指的是能夠指認該實體是獨一無二、可被明確辨識的個別存在。
- 例如，「小白」是鄰居的狗，有唯一身份；「人類」是整體分類，不具唯一身份。
- 「水韭潘的手機」是多台手機集合的分類，不是唯一的物件，因此是類別。

3. 屬性的判斷

- 屬性通常是「無法獨立存在的值」，用來描述物件的特徵。
- 例如，「顏值」是用來形容人的外貌吸引力的數值，是抽象的特徵。
- 「店家的地址」雖是可具體定位的資訊，但在此情境中是描述店家的特徵，故為屬性。

🧩 例題解析與應用

詞彙	是否實體	是否唯一	分類	判斷依據簡述
人類	是	否	類別	抽象分類名詞，無唯一個體形象
電池的電量	否	—	屬性	電量為電池特徵，不是實體
水球軟體學院的課程	是	否	類別	課程為抽象分類，不唯一
隔壁鄰居的小狗小白	是	是	物件	唯一存在的個體
太陽	是	是	物件	唯一且具體存在的天體
顏值	否	—	屬性	抽象特徵值
類別圖中的屬性	是	否	類別	抽象分類，非唯一實體
台灣	是	是	物件	唯一且具體的地理實體
軟體設計模式精通之旅（特定課程）	是	是	物件	唯一存在的課程
水韭潘的手機	是	否	類別	可有多台手機，非唯一
曖昧的程度	否	—	屬性	抽象特徵
水韭潘	是	是	物件	唯一且具體的個體
今天早上做的第二件事	是	是	物件	唯一指定的事件
事情的優先序	否	—	屬性	描述事件的特徵
店家的地址	是	否	屬性	描述店家特徵

🔍 分類判斷流程示範

以下以「隔壁鄰居的小狗小白」為例，展示 SOP 判斷步驟：

Thetawave.ai

1. **是否可實際存在的實體？**
   小白是鄰居家真正存在的狗 → 是。
2. **是否存在且唯一？**
   小白是特定且唯一的個體，而非所有小狗的總稱 → 是。
3. **結論**
   小白屬於**物件**。

🗣️ SOP 使用建議

- 大聲朗讀每個判斷問題，有助於理清思路。
- 確保答案有對應 SOP 的正確解釋，避免憑直覺或印象判斷。
- 持續練習可熟悉流程，快速判斷詞彙類型。

物件、類別與屬性的核心區別

特性	物件（Object）	類別（Class）	屬性（Attribute）
實際存在	是	是	否
唯一性	是	否	不適用
定義	獨特且具體的實體	抽象分類名詞，代表一組實體	描述實體或類別的特徵或性質
例子	太陽、水韭潘、小白、台灣	人類、水球軟體學院的課程、水韭潘的手機	電池的電量、顏值、曖昧程度、店家的地址

物件、類別和屬性之間的關係可比喻為一棟房子：

- **物件**是「具體的那一棟房子」，
- **類別**是「房子的種類（例如公寓、別墅）」，
- **屬性**是「房子的顏色、樓層數等特徵」。

這種關係強調物件的唯一存在性、類別的抽象歸類性，以及屬性描述的特徵性。

1.8 軟體抽象之二_萃取行為及操作.mp4

🧩 責任、行為、互動、操作的四步驟萃取

> 「透過精準區分『責任（Responsibility）』、『行為（Behavior）』、『互動（Interaction）』及『操作（Operation）』，能有條理地從需求中抽取出物件導向程式設計中類別的方法與函數，確保方法的正確性與設計的合理性。」

責任與行為的區分 🎯

名稱	定義	特徵與說明	範例
責任 (What)	高階描述，代表實體的定義及關係，說明「是什麼」或「負責什麼」	描述實體本身的存在意義與角色，包含它與其他實體的關係；屬於抽象層次較高的說明	象棋排堆的責任是由一堆牌所構成

Thetawave.ai

行為 (How)	具體動作，程式中需實作的步驟與規則，說明「怎麼做」	明確規範實體的功能流程與規則，必須用程式碼實現	課程平台自動計算學生經驗值；「將」只能水平或垂直移動一格

解釋

- **責任**類似於物件的「身份與存在理由」，它告訴你這個物件是什麼以及它該負責什麼。
- **行為**則是這個物件如何執行它的責任，是動態、具體且程式可實作的步驟。

具體範例：象棋需求中的責任與行為分析

句子	分類	說明
雙方棋子各自布置於棋盤兩側	責任	棋盤的責任描述，說明棋盤的狀態與關係
每種棋子的走法不同	責任	棋子的高階責任，定義不同棋子有不同走法
「將」只能在九宮格內水平或垂直移動一格	行為	具體規則，需程式碼實作
只能在九宮格內斜移動一格	行為	具體規則，需程式碼實作

互動與操作的萃取 🔄

名稱	定義	特徵與說明	範例
互動 (Interaction)	實體之間為完成責任而進行的協同行為	行為的一部分，代表多個實體間的交流與配合，單靠一方無法完成責任時才會產生互動	人在計算機上輸入數字、按運算子鍵、按等於鍵
操作 (Operation)	類別為提供能力而開放的方法或函數	從互動中萃取出的功能點，讓其他物件可以「呼叫」此方法並傳入參數，產生回傳結果或狀態改變	計算機類別的按鍵操作 (0~9數字鍵、加法、減法、乘法、等號)

操作格式（UML風格）

```
操作名稱（參數列表）：回傳型別
```

- 例子：
  `teach(student, book) : int`
  表示家教類別的操作名稱為 teach，傳入學生與書本兩個參數，回傳整數成績。

範例說明：人與計算機的互動與操作

互動內容	對應操作	行為描述
人在計算機上按下數字鍵 0~9	數字鍵操作（0~9）	將輸入的數字加入輸入序列
人按下運算符號（加、減、乘）	運算子操作（+、-、*）	更新計算機當前的運算狀態
人按下等號	等號操作	計算並顯示結果

Thetawave.ai

四個關鍵專有名詞整理

名稱	英文	定義與說明
類別	Class	需求中具有特定意義的實體
屬性	Attribute	類別所擁有的靜態特性
責任	Responsibility	類別所負責的「是什麼」或「做什麼」的高階描述
行為	Behavior	類別具體執行的動作和規則
互動	Interaction	多個類別間為完成任務所進行的配合
操作	Operation	由類別提供給外部呼叫的具體功能方法

實務應用：從需求到領域模型的萃取步驟

步驟	內容說明
1. 萃取類別	從需求中找出具有特定意義的詞彙，定義類別（物件）
2. 萃取責任	為每個類別定義其高階職責，描述它「是什麼」及「負責什麼」
3. 萃取屬性	針對類別補充靜態特質，如資料欄位
4. 萃取行為	具體描述類別需要執行的步驟與規則，為未來程式碼實作提供依據
5. 發現互動	找出類別間的互動關係，說明彼此如何配合履行責任
6. 萃取操作	依據互動內容，定義類別必須提供給外部使用者或其他類別呼叫的方法

單句分析練習技巧 ✍️

- **快速閱讀需求句子**，抓出關鍵詞和具意義的實體
- **先定義類別，接著萃取責任與屬性**
- **區分責任（what）與行為（how）**
- **找出行為中涉及的互動，萃取操作**
- **操作須包括名稱、參數與回傳型別**
- **保持專注於分析，不必急於考慮程式實作細節**
- **避免自行假設，必要時向需求提出者提問澄清**
- **多練習單句分析，養成習慣後能快速從完整需求中萃取模型**

典型案例比較表：玩家與怪物互動

類別	責任 (Responsibility)	行為 (Behavior)	互動 (Interaction)	操作 (Operation)
玩家	配合遊戲流程進行遊戲決策	受到攻擊時可能反擊	與怪物互動，接受攻擊並可能反擊	`beAttacked(damage: int)`，回傳空值；`counterAttack()`
怪物	遊戲虛擬決策體，主動攻擊玩家	攻擊玩家，受到攻擊時不反擊	與玩家互動，發起攻擊	`attack(player: Player)`，回傳空值

Thetawave.ai

UML 類別圖中行為與操作的表達方式

- UML 類別圖中**類別（Class）**會包含名稱、屬性（Attribute）、操作（Operation）
- **行為（Behavior）**雖無法直接內嵌於類別圖中，但可透過**便條紙（Note）**補充說明
- 操作格式示例：

```
操作名稱(參數1: 型別, 參數2: 型別) : 回傳型別
```

- 例：submitHomework(assignment: Assignment) : void 表示提交作業操作，無回傳值

物件導向分析（Object-Oriented Analysis，OOA）思路總結

- 由需求出發，透過點的萃取找到實體與類別
- 接著深入萃取類別的責任與屬性
- 再往下萃取具體行為，明確程式碼需實作的細節
- 行為中包含多實體間的互動，互動促成操作的產生
- 操作是外部呼叫的接口，讓類別可提供能力給其他類別或使用者
- 分離分析（OOA）與設計（Object-Oriented Design，OOD）兩階段，先聚焦需求的結構化，再專注程式運作與設計模式的套用
- 持續訓練單句分析可快速建立領域模型，提升需求理解與問題拆解能力

上述分析與萃取方法讓物件導向分析能夠清楚定義類別的方法與功能，並能有效區分抽象責任與具體行為，從而支撐後續的設計與實作階段。

1.9 軟體抽象之三_線的萃取.mp4

📏 線的萃取與關係概念

> **關係（Relationship）**是連結兩個或多個類別（Class）之間的線，代表它們之間的互動或結構連結。

- **線**代表的是類別間的關係，是物件導向分析（Object-Oriented Analysis, OOA）中不可或缺的元素。
- 一個系統的結構由點（類別）和線（關係）組成，理解線的種類與語義是掌握系統架構的關鍵。

🔍 關係的重要性與類型

為何關係重要？

- 以《航海王》劇本為例：理解角色及他們的相互關係，才能抓住劇情脈絡。
- 系統需求同理：必須萃取出重要實體（類別）及它們之間的關係，才能準確表達系統結構。

關係的五種分類（本課程重點前三種）

Thetawave.ai

關係類型	中文名稱	是否本課程重點	簡介
Dependency	依賴（動態關係）	是	只在行為執行期間存在的關係
Association	關聯（靜態關係）	是	行為之外也存在的關係，表示「有」的語義
Generalization	一般化/繼承（靜態關係）	是	「是一種」的語義，最強的結構關係
Aggregation	聚合（靜態關係）	否	複合關係的較弱版本（本課程暫不討論）
Composition	複合（靜態關係）	否	聚合的強版本（本課程暫不討論）

⚖️ 動態關係與靜態關係的區別

特性	動態關係（Dependency）	靜態關係（Association、Generalization）
關係存在時間	只在行為期間存在（執行時）	行為外也存在，長期存在
強度	較弱	較強
例子	喝水時用寶特瓶（口渴時的行為內發生關係）	老師「有」教室、一個物種是另一物種的子類（定義性關係）
符號	虛線箭頭，語義為 user（使用）	實線箭頭（關聯）或空心箭頭（繼承）

🕹️ 動態關係：依賴（Dependency）

> 「依賴」是指關係只在某個行為或過程中存在，行為結束，關係即消失。

依賴的判斷與表示

- 虛線箭頭表示依賴關係。
- 語義是「使用到」（user）。
- 依賴關係只能在行為中被發現，行為之外不維持。

例子分析

範例句子	類別抽取	行為	關係解析
口渴時用寶特瓶喝水	人、寶特瓶	喝水	喝水行為中人用到寶特瓶，產生依賴關係
「給我哑铃，锻炼三头肌」	人、哑铃、三頭肌	使用哑铃锻炼三头肌	行為中人依賴哑铃，產生依賴關係
「checkout购物车时产生订单」	購物車、訂單	checkout結帳產生訂單	行為中購物車需要知道訂單以創建訂單，為依賴

註：物件的「建立」也屬於依賴關係。

🔗 靜態關係之一：關聯（Association）

> 關聯表示兩個類別間長期存在的「有」（has）關係，即使無行為發生，關係依然成立。

關聯的特點與表示

- 實線箭頭表示關聯。
- 語義是「有」。

Thetawave.ai

- 有箭頭表示單向存取權，無箭頭則為雙向。
- 可能出現**遞回關聯**（自己指向自己），例如人追求另一個人。

例子分析

範例句子	類別抽取	行為	關係解析
老師帶領這堂課一整學期	老師、課堂	教導行為	老師與課堂間存在一學期的關聯（有）
「我永遠不放棄對消滅的追求」	人	追求行為	人與自己間的遞回關聯，語義為「有」
「動物園養很多動物」	動物園、動物	無特定行為	動物園與動物間為關聯，表示「動物園有動物」

👑 靜態關係之二：繼承（Generalization）

> 繼承表示「是一種」（is a）關係，是三種關係中最強的結構關係。

繼承的特點與表示

- 實線帶空心箭頭表示繼承。
- 語義是「是一種」，用於表示子類別與父類別的階層關係。
- 只能從定義（definition）中直接推導，無法從行為中發現。

例子分析

範例句子	類別抽取	行為	關係解析
獅子是动物的一种	獅子、动物	無	獅子繼承自動物，is a 關係
AI和人類都是參與者的子類	AI、人類、參與者	無	AI與人類繼承參與者，is a 關係

🎯 進階：關聯的基數（Multiplicity）

> **基數**描述關聯兩端各自能參與的實體數量，反映關係的約束條件。

基數的意義與表示方式

基數表示	含義	說明
1	必須有且只有一個	如「公司由一位老闆創辦」
0..1	可以沒有或有一個	如「人可能沒有結婚對象」
* 或 1..*	一個以上，可多個	如「課堂可有多位學員報名」
0..*	零個以上，可多個	如「人可能有零或多位朋友」
數字區間（如 0..52）	有明確上下限的範圍	如「牌堆最多有52張牌」

基數的判斷方法

- 針對關聯的每一端，思考該端一個實體能對應多少個另一端的實體。
- 例如：老師一學期可以帶幾堂課？一堂課可以被幾位老師帶？

基數的重要性

- 精確的基數有助於需求的正確理解與資料庫設計（Primary Key、Foreign Key）。
- 若基數標註錯誤，會導致系統設計及資料遷移（Data Migration）問題。

- 在程式設計中影響資料結構的選擇（如 List、Collection、Set 等）。

Thetawave.ai

🧩 推導關係的來源與流程

來源

關係只來自兩個地方：

來源	內容說明
需求中直接定義	例如「動物園有養獅子」、「老師是學校職員」等定義句子
行為中發現	透過行為分析發現關係，如「人吃飯時使用碗」、「玩家抽牌」

推導流程（SOP）

1. **從行為中發現關係：**
   - 判斷關係是否存在於行為以外。
   - 若只存在於行為中，即為依賴（Dependency）。
   - 若存在於行為外，為靜態關係，進入下一步分類。
2. **從定義中發現關係：**
   - 靜態關係只能從定義中直接推導。
3. **靜態關係分類：**
   - 語義是「is a」→ 繼承（Generalization）。
   - 語義是「has」→ 關聯（Association），需繼續分析基數。

🏷️ 符號與語義總整理

關係類型	符號	語義	存在時間	強度	來源
依賴 (Dependency)	虛線箭頭	使用（user）	行為期間	較弱	行為中發現
關聯 (Association)	實線箭頭/無箭頭	有（has）	行為外及行為中均可	中等	定義或行為中
繼承 (Generalization)	實線空心箭頭	是一種（is a）	行為外	最強	定義中發現

📚 例題綜合分析示範

句子：「动物园养多只动物，有狮子和长颈鹿」

- 類別：動物園、動物、獅子、長頸鹿
- 關係：
  - 獅子和長頸鹿皆繼承自動物（繼承，is a）
  - 動物園與動物間是關聯（有，has），表示動物園「有」多只動物
- 基數：
  - 動物園端：1..*（表示動物園可有多只動物）
  - 動物端：1（動物必屬於某動物園）

Thetawave.ai

💡 其他重要觀念

- **關聯方向表示存取權**：箭頭指向強調知道或能存取的方向，無箭頭表示雙向。
- **遞回關聯**：類別可以與自己產生關聯，例如「人追求另一個人」。
- **行為中發現的關聯**：不只依賴，也可能是關聯，需判斷關係強度與存在時間。
- **基數不標註或標錯會嚴重影響需求理解與後端實作**，代表設計者基礎功力與態度。

🛠️ 關係強度與系統設計啟示

- 耦合強度由弱至強：依賴 < 關聯 < 繼承
- 理想設計中耦合強度越低越好，避免過度繼承造成結構僵化。
- 需求分析階段準確抓出關係與基數，是後續良好架構設計與維護的基礎。

🧩 物件導向分析（OOA）的關係與建模思維

> **關係（Relationship）**是連結不同實體（Entity）之間的重要元素，協助形成整體架構與結構。

---

1. 關係的雙向基數（Cardinality）理解

問題方向	問題內容	結果說明
從 A 的角度看關係	一個 A 在同一時間內，可以與多少個 B 有此關係？	得到 B 端的基數（多寡）
從 B 的角度看關係	一個 B 在同一時間內，可以與多少個 A 有此關係？	得到 A 端的基數（多寡）

- **基數（Cardinality）**是關係中衡量一方可連結多少另一方的數量。
- 這種雙向的詢問是進行關係分類的標準作業程序（SOP）。

---

2. 物件導向分析中的關係角色

- 關係延伸了原先的分析視角，使得 OOA 不再只是**實體（Entity）**、**職責（Responsibility）**、**屬性（Attribute）與行為（Behavior）**，而是多出一層「關係」。
- 透過關係，可以連結不同的實體，建立出清晰的**架構（Architecture）或結構（Structure）**。
- 在類別圖（Class Diagram）的視角中，關係不僅是行為與互動的補充，更是多個類別間結構脈絡的核心。

---

3. 象棋案例：繼承與行為分配的思考

元素	說明	例子與意義
棋子（父類別）	定義共通行為	所有棋子移動後都要判斷「將軍」是否導致遊戲結束。

Thetawave.ai

不同棋種（子類別）	繼承棋子類別並分配特定行為	車、馬、炮、卒等，每種棋子有不同的走法驗證。
共同行為與特有行為的分配	共同行為寫在父類別，特有行為放在子類別	把共同行為集中管理，特定行為分散至子類別以降低複雜度。

- **繼承（Inheritance）**機制將共同行為集中，特殊行為分散，類別之間的關係更清晰且易管理。
- 分類成子類別通常能有效降低複雜度，但不一定每次都需拆分，視模型表達力與複雜度需求決定。

---

4. 模型複雜度與表達力的決策原則

決策重點	解釋	影響
是否拆分子類別	根據是否能降低複雜度判斷	在象棋案例中，分子類別明顯降低行為複雜度。
模型的表達力	以模型是否能有效幫助思考為優先考量	有時集中全部行為反而簡潔，拆分反而增加複雜度。
建模的目標	幫助自己更好思考需求與問題	建模決策應該服務於思考效率與清晰度，而非盲目追求複雜結構。

- **建模是思考的輔助工具**，不應為建模而建模。
- 在物件導向分析階段（OOA），不必過早將模型做過度複雜的拆分，後續的物件導向設計（OOD）階段才是更深入細化的時機。

---

5. OOA 的分析視角與階段成果

分析內容	說明
點（Entity）	實體，系統中的主要物件
線（Relationship）	實體之間的關係，連結不同點，形成整體架構
行為（Behavior）	實體或類別所執行的動作與反應

- 完成點、線、行為的萃取，代表 OOA 的核心概念已掌握。
- 後續進入需求分析與設計階段，將理論落實於實際系統建模。

關鍵觀念總結

- **關係的基數分析**是理解實體間互動的基礎。
- **繼承與子類別劃分**有助於管理行為複雜度，但需視具體情況判斷。
- **建模的最終目標是促進思考與降低複雜度**，而非過度細化模型。
- **OOA 聚焦於點、線、行為的萃取**，為後續物件導向設計奠定基礎。
- **多種模型表達方式並存**是正常現象，需根據需求靈活選擇。

物件導向分析的關係層次與繼承結構彼此交織，共同形塑系統模型的清晰架構與有效管理，從而提高系統設計與理解的效率。

1.10 兩種更強的關聯_聚合＆複合.mp4

🔗 聚合與復合關係

Thetawave.ai

> **聚合（Aggregation）與復合（Composition）**是物件導向分析中關係的升級版，屬於靜態關係，介於一般關聯（Association）和繼承（Inheritance）之間，強調「擁有」的強度及部分與整體的存在依賴。

---

1. 實體關係的強度排序

關係類型	強度排序	關係性質	說明	符號說明
依賴（Dependency）	最弱	動態關係	在行為互動中存在，虛線表示	虛線
關聯（Association）	一般	靜態關係	普通的「有」（has a）語義	實線
聚合（Aggregation）	較強	靜態關係	擁有者（owner）控制被擁有的物件（owns a）	空心菱形（主人端）
復合（Composition）	最強	靜態關係	整體與部分（whole-part）不可分離	實心菱形（整體端）
繼承（Inheritance）	最強	靜態關係	is a 語義，子類繼承父類	三角箭頭

🌟 聚合（Aggregation）

定義與語義

- **聚合**是一種「擁有（owns a）」關係，強調**擁有者對被擁有物件的主宰與控制權**。
- 被擁有物件可存在於多個擁有者之外，但只有擁有者有權控制、操作該物件。
- 這種關係表示「有」的強度比普通關聯更強，但部分物件仍可獨立存在。

判斷關鍵問題

> *是否只有 A（擁有者）能控制、操作或改變 B（被擁有者）的狀態？如果是，則為聚合關係。*

例子解說

例子	解釋
小明與跑車	小明擁有跑車，只有小明有權控制跑車，非經小明同意其他人不可操作。
汽車與四個輪胎	汽車擁有四個輪胎，汽車控制輪胎的狀態，其他汽車不可干涉。
股東與公司	多位股東共同擁有公司，控制權依股份比例分配，仍屬聚合關係。

符號表示

- 聚合使用**空心菱形**標示，菱形位於「擁有者（主人）」端。

🌟 復合（Composition）

定義與語義

Thetawave.ai

- **復合**是最強的「整體 - 部分（whole-part）」關係，強調**部分物件無法獨立存在，必須依附於整體物件**。
- 部分的生命週期完全依賴於整體，整體不存在則部分不存在。
- 這種關係代表物件間的強生命週期依賴與結構完整性。

判斷關鍵問題

> *如果 A（整體）不存在，B（部分）是否無法獨立存在？若是，則為復合關係。*

例子解說

例子	解釋
學校與教室	教室是學校的一部分，教室無法獨立存在於學校之外。
主面板與按鈕	按鈕依附於主面板，沒有主面板則按鈕無法存在。
訂單與訂單項目	訂單是整體，訂單項目是部分，訂單刪除時訂單項目也應被刪除。

符號表示

- 復合使用**實心菱形**標示，菱形位於「整體」端。

🔍 聚合與復合的符號與判斷整理

關係類型	強度	生命週期依賴	控制權	物件存在關係	符號
依賴	弱	無	無	無	虛線
關聯	較弱	無	無	無	實線
聚合	較強	無	有	可獨立	空心菱形
復合	最強	有	有	不可獨立	實心菱形

🛠️ 聚合與復合在實作上的影響

事項	聚合	復合
權限控管	強調只有擁有者可操作被擁有物件	強調整體擁有部分，部分不可隨意操作
物件生命週期管理	部分物件可獨立存在	部分物件生命週期依賴整體
刪除行為	部分物件刪除不一定影響整體	整體刪除時，部分物件必須同時刪除（瀑布式刪除）
垃圾回收影響	較少，因部分可獨立	無垃圾回收語言需特別注意物件銷毀

💡 重要思考技巧

- **設計決策時，聚合與復合的關鍵判斷問題非常重要**，不必過度糾結初期就決定，先以「一般關聯」標示，待有更多資訊時再決定是否升級。
- 符號的空心與實心菱形不只是形狀差異，反映了關係強度與語義，**空心象徵較弱的擁有權，實心象徵強烈的整體 - 部分依賴**。
- 生命週期與控制權是聚合與復合最大的區別點。

Thetawave.ai

📌 符號記憶法

關係類型	符號	記憶技巧
依賴	虛線	最弱動態關係，用「虛線」表示不確定性
關聯	實線	基本靜態關係，實線代表存在的連接
聚合	空心菱形（主人端）	「空心」代表擁有但較弱的控制權
復合	實心菱形（整體端）	「實心」代表整體與部分緊密結合

聚合與復合區分了物件間的擁有權與生命週期依賴，聚合注重控制權，部分可獨立；復合則強調整體與部分不可分割的存在關係，部份物件生命週期依賴整體。這種區分幫助在物件導向分析中精確表達實體間的強度差異與管理策略。

1.11 線上的點_關聯類別.mp4

🔗 什麼是「關聯類別」（Association Class）

> **關聯類別**指的是**存在於兩個類別之間的「關聯」本身所具有的屬性或行為**，它是「關聯」的升級版，是「線上的點」，能在模型中捕捉原本只能表示為關係的額外資訊。

關聯類別的產生條件

在 UML（統一建模語言）中，當一個「關聯」滿足下列任一條件時，可以將該關聯「升級」成為**關聯類別**：

條件編號	條件說明
1	關聯有衍生的屬性需要被記錄
2	關聯本身有行為，且該行為必須以關聯存在為前提

> **關鍵點：必須先有關聯，然後才決定是否升級為關聯類別。**

🧩 解析關聯類別的案例說明

案例一：學生與課程的「期末考成績」

- **背景需求**：學生上每一堂課會有一個期末考成績
- **分析步驟**：
  1. 先有兩個類別：「學生」與「課程」
  2. 學生「報名」課程，形成「上課」的關聯
  3. 期末考成績是一個屬性，但它不是學生或課程本身的屬性，而是「上課」這個關聯所衍生的屬性，因為學生必須先上課，才會有成績
- **結論**：
  - 期末考成績符合條件1（關聯有衍生屬性）
  - 將「上課」關聯升級為關聯類別，捕捉期末考成績屬性
- **符號表示**：
  - 在關聯線上用虛線連接一個類別框，該類別即為「關聯類別」
  - 關聯類別可以有屬性、操作（行為）

Thetawave.ai

案例二：兩人之間的「婚姻」及「離婚」行為

- **背景需求**：兩人相愛而結婚，結婚後可離婚
- **分析步驟**：
  1. 類別「人」
  2. 人與人之間有「婚姻」的靜態關聯
  3. 「離婚」是行為，但必須在有「婚姻」關聯存在下才能執行
- **結論**：
  - 「離婚」行為符合條件2（關聯本身有行為且依賴關聯存在）
  - 將「婚姻」關聯升級成為關聯類別，捕捉「離婚」行為
- **重要理解**：
  - 行為（操作）不一定屬於單一類別，有可能屬於兩類別之間的關聯

案例三：城市之間的「道路連結」與「路名」

- **背景需求**：任兩個城市可能由一條道路連接，每條道路有路名
- **分析兩種建模方式**：

方式	描述	優缺點及說明
方式一	將「城市」與「道路」視為兩個不同類別	「道路」是獨立類別，擁有屬性「路名」
方式二	將「道路連結」視為「城市」間的關聯	將「道路連結」升級為關聯類別，捕捉「路名」屬性

- **關聯基數分析**：
  - 每個城市可連接 0 到多個城市
  - 關聯的兩端都是 0..*（多重性）
- **結論**：
  - 不同建模思路導致不同表現形式
  - 兩種模型在實作上等價
  - 根據需求與分析角度選擇表達方式即可

🧠 重要概念與容易混淆點

主題	說明
關聯 vs. 關聯類別	「關聯」是兩類別間的關係；「關聯類別」是帶有屬性或行為的關聯，表現為線上的類別。
關聯升級條件	只有符合「關聯有衍生屬性」或「關聯有行為且依賴關聯存在」其中一項，才能升級為關聯類別。
屬性放在哪裡	若屬性屬於兩個類別間的關聯，應放在「關聯類別」而非單一類別中。
行為的歸屬	行為可以屬於類別，也可以屬於關聯（如離婚行為必須存在婚姻關聯才能執行）。
模型切入點不同	同一需求可有不同建模方法，重點是分析有據，避免陷入「哪種方式才是正確」的選擇障礙。
線上的點	「關聯類別」即為「線上的點」，是結合「點的有限性」與「線的連結性」後的特殊元素。

Thetawave.ai

📐 UML 關聯類別符號與表示

元素	說明	UML 符號示意
類別（Class）	實體物件的抽象表示	實心矩形框（含名稱、屬性、行為）
關聯（Association）	兩類別間的連結	實線連接兩個類別
關聯類別（Assoc. Class）	關聯本身的類別，帶有屬性與行為	實線關聯上方虛線連接的類別框

UML 中的 6 種關係類型

序號	關係名稱	說明
1	泛化（Generalization）	表示繼承與父子類別關係
2	實現（Realization）	類別實作介面
3	組合（Composition）	強擁有關係，部分與整體生命週期相同
4	聚合（Aggregation）	較弱的擁有關係，部分與整體生命週期不同
5	關聯（Association）	類別間的結構性連結
6	**關聯類別（Association Class）**	**關聯上的類別，帶有屬性或行為的特殊關聯**

模型設計實務建議

- **先找關聯，再決定是否升級為關聯類別**，別急著直接定義關聯類別。
- **屬性和行為的歸屬需明確**：關聯性質的屬性和行為放在關聯類別，不屬於單一物件的屬性不要放錯類別。
- **靈活運用不同建模方式**，根據需求與分析情境選擇最適合的表達法。
- 避免糾結「哪種表示才是正確」，重點是**清晰表達需求與關係的本質**。
- 關聯類別提升模型的表達力，使結構模型更精確且具彈性。

關聯類別結合了實體類別與它們間的關係，形成一種既能描述物件本身又能捕捉物件間互動細節的結構，有助於提升系統分析與設計的精度與完整性。

1.12 軟體抽象之四_區分結構.行為和侷限.mp4

🤿 軟體抽象訓練與OOA三副眼鏡

> **軟體抽象**是指在腦中建立秩序，讓需求分析到程式設計的過程更有條理、精準，並非一開始就寫程式，而是先用抽象思維準備好架構。

軟體抽象的意義

- 軟體抽象能力是駕馭程式碼的基礎。
- 透過抽象，可以提早思考需求，避免寫程式時才發現問題。
- 抽象使需求在腦中形成有序的思維形狀，幫助快速映射成類別圖（Class Diagram）。

Thetawave.ai

OOA的三副眼鏡：結構、行為與局限

1. 結構（Structure）

- 由需求中**實體（Entity）**與它們之間的**關係（Relationship）**構成。
- 抓取需求中的「點」（實體）和「線」（關係）。
- 例子：象棋中的棋子、棋盤，玩家、遊戲板、格子等。

2. 行為（Behavior）

- 描述各類別會做的事情、執行的步驟。
- 使模型「動」起來，類別間開始互動。
- 例子：象棋中棋子的移動、玩家的抽牌、遊戲中輪流落子。

3. 局限（Constraints）

- 模型中必須**始終遵守的條件**。
- 有助於程式的防衛性設計與維護性。
- 總共三種主要局限：
  - **行為的前提（Preconditions）**：執行某行為前必須符合的條件。
  - **屬性值域（Value Range）**：屬性可以接受的值範圍。
  - **關係的前提（Relationship Preconditions）**：建立某關係的必要條件。

局限的詳細分類與說明

局限類型	說明	代表範例	實作說明
行為的前提	行為執行前必須符合的條件	象棋棋子移動必須遵守移動規則	在行為便條紙上，用大括弧標示行為前提，放置於行為描述上方
屬性值域	屬性可接受的合法值範圍	棋子顏色只能是黑或紅（列舉值域） 年齡必須介於18至120歲	YAML中於屬性型別後加大括弧描述值域 如列舉型別Enum或數字範圍
關係的前提	建立關係的條件	員工資歷>3年才可管理下屬 只有老闆可以沒有上級	於關係便條紙用大括弧標示前提
唯一值	屬性必須唯一	會員流水號、身份證字號、Email等必須唯一	在需求中標示「必須唯一」
多屬性不變條件	多個屬性間的關係必須保持不變	訂單總價=單價×數量 距離=速度×時間 面積=長×寬	把多屬性間的關係條件明確列出
格式限制	屬性格式限制	Email格式必須符合電子郵件規範 密碼必須包含大寫字母、數字、特殊字元	屬性定義型別為字串，後方用大括弧寫明格式規則
基數限制	資源數量的限制	帳密字元需介於16到25個字元	使用基數描述資源數量限制，如 Char{16..25}
初始值	屬性的預設初始值	使用者狀態預設為「正常」	屬性後方加等號設定初始值
時間範圍限制	屬性或事件的時間必須符合一定條件	活動日期必須大於今日 出貨時間必須早於現在 預約時間需提前兩小時	需特別注意時間值域是常被忽略的限制

實例說明與應用

Thetawave.ai

象棋案例

- **結構**：棋子、棋盤、玩家，棋子顏色（分黑紅）
- **行為**：棋子移動、玩家輪流操作
- **局限**：
  - 行為前提：移動必須符合棋子移動規則。
  - 屬性值域：顏色只能是黑或紅。
  - 關係前提：棋子關係必須符合棋盤範圍。

員工管理案例

- 屬性：員工資歷為整數，必須大於0。
- 關係前提：
  - 只有資歷超過3年者可管理下屬。
  - 只有老闆可無上級，員工上級基數為0或1。

抽牌遊戲案例

- 結構：玩家與牌，牌有「鬼牌」屬性（Boolean）
- 行為：玩家抽牌
- 局限：玩家手牌中不能有鬼牌（關係前提）

圈圈叉叉遊戲案例

類別	屬性與限制	行為與前提說明
玩家	符號只能是「圈圈」或「叉叉」	必須輪流下子，只能在空格子放子，且只能在自己回合
格子	符號可為空格、圈圈或叉叉	無直接行為，但影響遊戲結果
棋盤	有9個格子	遊戲結束條件：任一玩家連成三子或格子全滿平局

🥽 需求分析的核心技巧：單句分析與三眼識別法

- **單句分析**：一次分析一句話，避免信息爆炸。
- **三副眼鏡輪流戴**：
  1. **結構眼鏡**：找出類別與屬性。
  2. **行為眼鏡**：找出系統中發生的動作與過程。
  3. **局限眼鏡**：找出必須遵守的條件與限制。

🥽 屬性格式完整表達

屬性的定義格式如下：

```
名稱 ： 型別 [基數] = 初始值 { 局限 }
```

部分	說明	範例
名稱	屬性名稱	color

Thetawave.ai

型別	基本資料型別	Integer、String、Boolean
基數	資源數量限制（可選）	{16..25}（表示字元長度限制）
初始值	屬性的預設值（可選）	= 0、= "正常"
局限	包含值域、格式、唯一性等限制	{0,1}、{must be unique}

🤿 小結判斷

結構、行為與局限三者間相互依存，結構定義了系統的靜態框架，行為賦予系統動態功能，而局限則是維繫系統正確性與穩定性的規則。唯有準確區分並明確表達這三者，才能建構出可維護且符合需求的軟體模型。

1.13 Astah_類別圖教學.mp4

🖥️ Asta UML Editor 使用教學

> **Asta UML Editor** 是一款用來繪製 UML（統一建模語言）圖的工具，強調「行雲流水」的操作原則，即流暢且高效率地完成建模工作。

專案與畫布的建立

步驟	操作說明	重點提示
開啟新專案	點擊左上方的 New File 圖示	專案目錄(Project Root)會出現
創建新圖	在專案目錄上右鍵，選擇 Create Diagram	選擇類別圖（Class Diagram）開啟畫布

類別（Class）的建立與命名

建立方式	操作細節	使用建議
方法一	左上方點選類別圖標，再點畫布左鍵	適合單次建立
方法二（更快）	直接在畫布上雙擊	適合快速大量建立類別，配合需求分析節奏

- 點兩下類別名稱即可重新命名，例如命名為 Layer。

屬性（Attribute）與操作（Operation）的設定

操作項目	操作方法	注意事項與範例
開啟屬性視窗	將滑鼠移至類別上，點選橘色按鈕	輸入屬性名稱與型別，例如 Name : String
開啟操作視窗	點選綠色按鈕	輸入操作名稱，例如 MakeDecision()
設定能見度	左下方視窗設定 Visibility	一般屬性通常設為 Private，操作可依需求調整
設定初始值	左下方 Initial Value	例如 Name 初始值可設定為 "Johnny"，Age 初始值為 13
設定屬性約束條件	左下方 Constraint Tab 新增約束	例如 Age > 0，或 Email 屬性需符合 Email 格式

靜態屬性標示	左下方 Static 設為 True	靜態屬性名稱下方會有底線表示

Thetawave.ai

屬性基數（Multiplicity）與格式限制

- **基數**在 UML 中指的是元素的數量限制，通常用中括號表示，例如字串長度限制。
- 例子：
  - ID : String [10] 表示 ID 必須有 10 個字元。
  - 可設定為範圍，如 [0..10] 表示接受 0 到 10 個字元。

抽象類別與介面（Interface）

類別設定	操作說明	視覺效果
設定抽象類別	選擇類別，左下方將 Abstract 設為 True	類別名稱會變成斜體字
轉換為介面	類別上右鍵選 Convert to Interface	類別上方會多一個括弧尖角標示「Interface」

- **Stereotype（刻板印象）**：用於標示類別的類型，例如 Interface 或 Enum，透過右鍵選單或快捷鍵設定。

列舉型別（Enum Type）的建立與成員表示

方法	操作說明	補充說明
屬性與約束方式	在屬性中設定屬性名稱並限制值域（如花色）	約束值域為「梅花、方塊、愛心、黑桃」等
宣告新類別並標示為 Enum	新建類別，右鍵設 Stereotype 為 Enum	以類別呈現 Enum，成員用屬性表示，屬性型別可隱藏
隱藏屬性型別	右鍵類別選 Extended Visibility，取消勾選 Attribute Type	讓 Enum 成員屬性型別不顯示，視覺更清爽

類別隱藏區域與顯示設定

項目	操作方法	功能說明
隱藏屬性區域	右鍵類別選 Attribute Compartment Visibility 取消勾選	隱藏所有屬性顯示
隱藏操作區域	右鍵類別選 Operation Compartment Visibility 取消勾選	隱藏所有操作顯示
隱藏操作參數型別	右鍵類別選 Extended Visibility，取消勾選 Operation Parameter Type	不顯示操作參數的型別
隱藏操作回傳值型別	右鍵類別選 Extended Visibility，取消勾選 Return Type	不顯示操作回傳值型別

行為（Behavior）的建立與連結

Thetawave.ai

- 行為以「變條子」形狀的符號表示。
- 建立方法：點選上方變條子圖示，在畫布點擊生成，再將變條子用虛線連接至類別或操作。
- 連結說明：
  - 連到類別表示類別的職責（What）
  - 連到操作表示操作的具體行為（How）

關係（Relationship）的使用

關係類型	圖示說明	使用說明
依賴（Dependency）	虛線箭頭	選擇依賴圖示，從起點類別拉到終點類別建立
連結（Association）	實線（無箭頭為雙向，有箭頭為單向）	選擇連結圖示後拉線，可設定單向或雙向
連結別名（Alias）	關聯線上端點旁可設定別名	用於對關聯端命名，如將 card 別名為 HandCard
連結基數（Multiplicity）	米字號圖示，設定數量限制	例如 0..多，也可自訂具體數字如 0..108

連結方向與調整

- **方向性（Navigation）**決定存取路徑：
  - 單向關聯：A 可存取 B，B 不可存取 A
  - 雙向關聯：A、B 互相可存取
- 調整方法：
  1. 選擇關聯線
  2. 左下方選擇對應端（Association and A 或 B）
  3. 找 Navigation 選項，取消勾選 Navigable 或設定為 Unspecified
- 關聯線的箭頭僅提示拉線方向，無特殊語義。

連結別名與名稱設定

- 連結可以命名，例如 擁有（Ownership）。
- 命名後，關聯線會顯示黑色三角形箭頭指示起點至終點方向。

連結類別（Association Class）

- 連結類別是加強版連結，突出一個類別存在於連結上。
- 建立方法：選擇關聯類別圖示，拉線連接兩個類別，會在關聯上產生一個類別節點。
- 可轉回一般連結或類別：
  - 右鍵點選連結類別，選 Convert to Association 退回成連結
  - 選 Convert to Class 可退回成一般類別，等價於持有雙方引用

Thetawave.ai

連結約束（Constraint）

- 約束表示關係成立的條件。
- 建立方法：
  1. 拉出變調字（Note）
  2. 編輯約束條件文字，例如「不可以擁有鬼牌」
  3. 用虛線連接變調字與關聯線

排版工具與繪圖技巧

工具名稱	功能說明	使用情境
直角折線箭頭	取代斜線箭頭，讓繼承線更整齊美觀	建議80%使用以提升圖面清晰度
Gap Expander	將水平線以下的類別往下延伸，製造間距	批次調整類別間距，改善排版布局
Gap Remover	收回空白區域，縮減類別間距	將類別群組收攏，節省畫布空間
其他線條與符號	可拉出非 UML 原生但實用的線條符號	用於特定需求或自訂符號

套件（Package）的建立與管理

- **Package** 用於隔離並組織多個領域模型或需求。
- 建立方法：
  1. 點選左上方 Package 圖示
  2. 點擊畫布新增 Package 範圍
  3. 將多個類別全選後拖入 Package 中
- 移動 Package 時：
  - 內部類別會隨著移動
  - 其它符號（如變調字、箭頭）不會自動跟隨
- 解決方式：
  - 右鍵 Package 選 Depth Arrangement → Send to Back
  - 全部選取內容一起移動

快捷鍵建議

作業項目	Mac 快捷鍵	Windows 快捷鍵	功能說明
新增屬性	Command + R	Ctrl + R	快速開啟屬性編輯視窗
新增操作	Command + M	Ctrl + M	快速開啟操作編輯視窗

- 熟悉快捷鍵可大幅提升建模效率，讓建模速度跟上需求分析節奏。

UML 編輯器使用心得

- UML 編輯器是促進思考與溝通的工具，不是編譯器（Compiler），因此建模重點在於清晰表達與易讀性。
- 建模時應依照需求做顆粒度的調整，適度隱藏不必要資訊，強化關鍵內容。
- 透過靈活設定屬性、操作、關聯及約束，能更精確地描述系統架構與行為。
- 排版及視覺管理功能能幫助建立整潔易懂的模型圖，利於團隊協作與交流。

Thetawave.ai

1.14 練武 - 單句分析(初級).mp4

📝 單據分析的基本思路與步驟

> **單據分析**是將需求中具備意義的詞彙（點）和詞彙間的關係（線）萃取出來，形成物件導向分析的基礎，並區分結構、行為與限制，從而完整描繪需求。

單據分析的整體流程

步驟	說明
1. 點的萃取（詞彙辨識）	找出需求句子中所有**具特定意義、重要的詞彙**，判斷其是否為**實體類別（類別）或是屬性（描述實體特徵的值）**。
2. 行為與限制的區分	對非實體、非屬性的詞彙判斷為**行為（系統基於職責的動作）或限制（局限性）**。
3. 線的萃取（關係辨識）	在需求中定義的實體間關係或由行為中發現實體關係時，萃取關係並分類（靜態或動態關係）。
4. 參與者數量（基數）分析	對關係兩端的實體，分析其參與數量（最小與最大）以描繪多重性。
5. 升級關係為關係類別（若有）	若關係本身具備屬性或行為，將其轉換為**關係類別（association class）**，使關係能被具體描述。

📌 點的萃取：類別、物件與屬性判斷

詞彙種類	判斷標準與說明	範例
類別（Category）	抽象分類的名詞，代表一類實體，且可實際存在。	人、杯子、動物園、狗、貓、老師
物件（Instance）	獨一無二、已存在的實體，屬於某類別的具體個體。	小鬼（是狗的物件）、小白、小美
屬性（Attribute）	用以描述實體特徵的值，非獨立存在的實體。	名字、學期（課程的屬性）、結訓成績
行為（Behavior）	系統中基於職責或需求所做的事情，非實體或屬性。	喝水（人的行為）、當兵（人與軍營間的關係）

🔗 線的萃取：關係的分析與分類

兩個主要時機點

1. **需求中明確定義實體間的關係**
2. **行為中發現實體間的關係**

關係分類判斷流程 SOP

Thetawave.ai

問題	判斷依據與結果說明
需求中定義的關係是否存在？	是 → 靜態關係；否 → 從行為中找是否有動態關係
該關係屬於 is-a 還是 has-a？	is-a 表示繼承，has-a 表示關聯（aggregate/association）
是否有生命周期相依關係？	是 → 表示聚合（aggregation）或複合（composition）
參與者數量（基數）為何？	問兩個方向：每一端最少與最多的數量，可能為具體數字、星號（多）、問號（不確定）

關係的強度分類

關係類型	說明	例子
關聯 (Association)	一般的 has-a 關係，無生命周期依賴	老師與課程、家與狗、動物園與狗、動物園與貓
聚合 (Aggregation)	部分與整體關係，部份可獨立存在	（未明確範例，但屬於較強關聯）
複合 (Composition)	部分與整體關係，部份生命周期依賴整體，不可獨立	套房與浴室（浴室不可獨立存在）
繼承 (Inheritance)	is-a 關係，類別間的泛化	動物是長頸鹿與蛇的泛化類別

🎯 重要概念與實例解析

1. 結構 vs 行為 vs 局限

- **結構**：指實體（類別）與其屬性，是系統靜態部分
- **行為**：系統中實體負責執行的動作或職責
- **局限**：系統的限制條件（本教材未深入）

範例：「人在喝水時用到杯子」

- 「人」、「杯子」為結構（類別）
- 「喝水」為行為，且行為中產生了「依賴」關係（人依賴杯子）

---

2. 物件與類別的區分

- 物件是類別的實例，不可將物件誤當成類別

範例：「小鬼是你家的狗」

- 「狗」是類別
- 「小鬼」是物件（實際存在且唯一），屬於「狗」類別
- 名字是「狗」的屬性

---

3. 基數（Multiplicity）分析

- 針對關係兩端，問兩個問題：
  1. 某端實體對另一端實體的最少與最多數量
  2. 反向同樣問題

範例：「老師每學期至少教 5 堂課」

- 老師端基數：至少 5（5 到 多）

- 課程端基數：未知（1 到？）

Thetawave.ai

---

4. 關係類別的升級

- 當關係本身具有屬性或行為，需將關係升級為**關係類別（association class）**

範例：「當兵」關係帶有「結訓成績」屬性

- 將「當兵」關係轉換成關係類別
- 「結訓成績」為該關係的屬性

📊 範例分析摘要表

需求句子	點（類別/物件/屬性）	行為	關係類型	基數（範例）	備註
人在喝水時會用到杯子	類別：人、杯子	行為：喝水	動態依賴關係（has-a）	人依賴杯子（動態）	依賴用虛線表示，線名為「喝水」
動物園裡有很多狗和貓	類別：動物園、狗、貓	無	靜態關聯	動物園到狗：*（多）	狗、貓只能在一個動物園或多個？不確定
你家那隻狗的名字叫小鬼	類別：家、狗	無	靜態關聯	家與狗：？到？	「小鬼」是狗的物件，名字是屬性
動物有很多種，長頸鹿和蛇算一種	類別：動物、長頸鹿、蛇	無	繼承（is-a）	-	繼承關係用實線箭頭表示
老師每學期至少要教 5 堂課	類別：老師、課程	無	靜態關聯	老師到課程：5 到 *	「至少」表示下限為 5
小白和小美是好朋友	物件：小白、小美（屬於人）	無	迴圈關聯（人與人）	0 到 *，雙向	關係為「好朋友」的關聯
排堆裝著至少 100 張撲克牌	類別：排堆、撲克牌	無	靜態關聯	排堆到撲克牌：100 到 *	撲克牌在 0 到 1 個排堆（常識判斷）
套房有兩間浴室	類別：套房、浴室	無	複合關係（composition）	套房到浴室：2 到？，浴室到套房：1	浴室依附套房生命周期，浴室不可獨立存在
當兵時會在所屬軍營留下結訓成績	類別：軍營、人	行為：當兵	靜態關聯（升級為關係類別）	軍營到人：1 到 *，人到軍營：0 到？	結訓成績為當兵關係類別的屬性

🎯 實務應用要點

- **堅持 SOP 流程，避免憑直覺判斷**，尤其初學時要一步步跟著流程走。
- **點的萃取先行**，不論句子多口語化或無序，都先找出所有具意義的詞彙。
- **行為與關係需細心區分**，動態關係來自行為，靜態關係來自需求明確定義。
- **物件與類別不同**，千萬不要將物件誤當類別。
- **基數分析不可省略**，要問雙向的最少與最多，若不確定先用問號標示。
- **關係類別的使用提高模型精確度**，當關係本身有屬性或行為時務必升級。
- **對不確定或未明述的部分勇於標註未知**，並在後續與需求提出者或領域專家確認。

模型與需求間的關聯性判斷

Thetawave.ai

物件導向分析中，**類別、物件、屬性、行為與關係**相互關聯，形成系統的靜態結構與動態行為。結構元素（類別與屬性）定義系統的骨架，而行為與關係描繪實體間的互動與依賴。透過正確分類與基數分析，模型能有效反映現實需求，並在需求演變時保持清晰與彈性。這種分析方法不僅有助於需求理解，也為後續系統設計與實現打下堅實基礎。

🏢 物件導向分析中的類別與關聯類別

> 「類別之間可以建立關聯類別，以記錄1到多個額外的屬性（Attribute）或行為（Behavior）。」

---

1. 重要概念解析

關鍵詞	定義與說明
類別（Class）	在類別圖（Class Diagram）或領域模型（Domain Model）中，代表一種具有共通特徵的抽象分類，實際存在的實體（Entity）或事物的類型。類別本身是抽象的分類名詞，不代表唯一性身份。
關聯類別（Association Class）	一種特殊的類別，用來表示兩個類別之間的關聯（Association）且同時包含該關聯的屬性和行為。關聯類別兼具「類別」與「關聯」的雙重身份。
屬性（Attribute）	描述類別或關聯類別的特徵，是類別的內部資料。屬性本身也是一種抽象的分類，但通常不可辨識唯一身份。
行為（Behavior）	類別或關聯類別可執行的動作或功能，類似方法或操作，也是類別的一部分。

---

2. 類別與關聯類別的關係

- **關聯類別是類別的一種**，同時它承擔「關聯」的角色，連結兩個或多個類別。
- 當兩個類別之間的關聯需要記錄額外屬性或行為時，就會使用關聯類別。
- 關聯類別可以包含多個屬性與行為，而這些屬性與行為是描述該關聯本身的。

---

3. 類別之間的關聯語義及基數（Multiplicity）

關聯兩端	基數範圍	說明
人（勞工）→公司	0或1（可能多於1，表示兼職）	一個人可以不在任何公司工作（失業，0個），也可以在一個或多個公司工作。
公司→勞雇契約	1至多個（不確定最少是否為1）	公司可有多個勞雇契約，表示有多位員工。
類別→關聯類別	0到多（不確定最大數量）	類別之間可以有多個關聯類別，且非必要（可為0）。
關聯類別→屬性與行為	1至多個	一個關聯類別至少有一個屬性或行為，且可有多個。
屬性與行為→關聯類別	1個	屬性與行為必須屬於一個關聯類別，無法獨立存在。

---

4. 具體範例說明：勞雇契約模型

- **類別：公司**

Thetawave.ai

- **類別：勞工（人）**
- **關聯：勞雇契約（Association Class）**
  - 勞雇契約是公司與勞工之間的關聯類別，表示雙方間的契約關係。
  - 勞雇契約擁有屬性：基本月薪（整數，且不可為負數），其他條款（字串陣列，代表多條條款）。
  - 這些屬性是描述勞雇契約本身的特性，屬於關聯類別的屬性。

勞雇契約模型關係簡表

元素	說明	基數	屬性範例
公司	企業實體	1至多個勞雇契約	-
勞工	個人實體	0或多個勞雇契約（兼職可能）	-
勞雇契約（關聯類別）	公司與勞工間契約	1至多個	基本月薪（整數，≥0）、其他條款（多條字串）

---

5. 屬性與行為在類別與關聯類別中的位置

- **屬性與行為都屬於類別的特性**，它們不能獨立存在，必須依附於某個類別或關聯類別。
- **關聯類別可擁有屬性與行為**，用來描述兩個類別間關聯的額外資訊或功能。
- 若屬性與行為被定義在關聯類別中，表示其意義是專屬於該關聯的，而非單一類別的特徵。

---

6. 分析與思考流程建議（SOP）

- **抓出重要名詞**：類別、關聯類別、屬性、行為。
- **釐清詞義**：理解類別是抽象分類，關聯類別是同時具備類別和關聯性的特殊類別。
- **釐清關係與基數**：判斷兩端是否可有多個實例、是否必要存在。
- **歸類屬性與行為**：判斷這些特徵屬於哪個類別或關聯類別。
- **模型調整**：必要時將關聯類別轉換為類別，或設計繼承結構以完善模型。

---

7. 類別圖中常見結構比較

元素	可存在唯一身份	是否為抽象分類	是否可獨立存在	可包含屬性與行為
類別	否（代表一類）	是	是	是
關聯類別	否（表示關聯）	是	是（因為是特殊類別）	是
屬性	否	是	否	否
行為	否	是	否	否

---

8. 類別與關聯類別的繼承（擴充觀點）

- **關聯類別自身也是一種類別**，可以視為類別的子類型或特殊化。
- 若模型中加入「關聯類別是類別」的語句，則需在類別圖中建立繼承關係，提升模型完整性與精確度。

- 屬性與行為必須歸屬於某個類別（包含關聯類別），不可獨立存在，這意味著類別與關聯類別的結構需設計成嚴謹的組合（Composite）關係。

Thetawave.ai

---

9. 執行範例 — 類別圖的關聯類別轉換

- 當發現屬性和行為屬於兩個類別之間的關聯時，將關聯轉換成「關聯類別（Association Class）」。
- 透過關聯類別，可以將屬性與行為放入該關聯，清楚表達該關聯的特性與功能。
- 例如：將「基本月薪」與「其他條款」從勞雇契約的描述屬性轉換成關聯類別的屬性。

---

10. 思路的重要性

- 面對抽象且複雜的描述，依照SOP（標準作業流程）拆解關鍵詞與概念，是理解物件導向分析的核心。
- 明確掌握類別、關聯類別、屬性、行為的定義與相互關係，能使模型設計得更精確與有表達力。
- 在不確定細節時，可用基數範圍「0..1」、「0..多」等表示彈性，並標註問號表示待釐清，保持模型彈性且不失嚴謹。

物件導向分析中，類別與關聯類別的設計彼此環環相扣，透過關聯類別的引入能有效捕捉類別間的複雜關係，並以屬性與行為豐富其語義，這種結構使得模型不僅完整且具備可擴充性，對後續的系統設計與實作具有深遠影響。

1.15 練武 - 單句分析(進階).mp4

🧙‍♂️ 行為與物件導向分析基礎

> 「行為（Behavior）指的是系統中實體所執行的動作或互動，是分析過程中的核心概念。」

重要分類詞彙萃取

分類詞彙	說明
法師	主體角色，會執行放招行為
魔力	法師的屬性，數值範圍從0到多（最大值未定）
怪物	對象，法師放招的目標，會被打（被打是怪物的操作）

行為與互動的區分

- **行為**：超出分類詞彙之外的描述皆視為行為，例如「法師放招打怪物」。
- **互動（Interaction）**：行為中涉及多個實體間的合作或能力使用。例如「打怪物」需怪物提供「被打」的能力。

Thetawave.ai

互動中操作的萃取

- 行為包含互動時，需於相關類別中開出對應操作。
- 例：法師打怪物，怪物需有「被打」操作，代表怪物被攻擊的能力。

行為推導關係線的原則

問題	答案	說明
行為外是否存在該關係？	只存在於行為裡	例如「放招打怪物」關係只在放招行為中出現，因此是依賴關係
關係來源	行為中發現	關係線不是直接從需求定義，而是由行為衍生

🎮 遊戲流程中的類別與行為建模

範例：遊戲開始與玩家抽牌

類別	職責說明	行為描述
遊戲	控制整體流程	遊戲開始
玩家	做出決策	抽四張牌
牌	遊戲資源	無直接行為

- 玩家與牌之間的**關聯**：玩家可擁有4張牌（基數確定），牌只屬於一位玩家。

行為與關係的歧異解讀

- 行為歸屬可依責任分配不同解讀：
  - 「玩家抽牌」可視為玩家行為，也可視為遊戲控制流程的行為。
- 建議依照職責劃分：
  - 遊戲控制流程 → 遊戲行為
  - 玩家做決策 → 玩家行為

行為中互動操作的可能遺漏

- 例如玩家「獲得手牌」操作，可能在分析階段未被立即發現。
- 80%完整度為合理起點，未來設計階段（物件導向設計，Object-Oriented Design，OOD）可補足。

🏆 比賽與主持人模型

類別	職責說明	行為描述
比賽	控制比賽流程	開始比賽
主持人	執行實況轉播	實況轉播

- 比賽與主持人間存在需求關聯（每場比賽有一位主持人）。
- 行為中主持人被要求執行「實況轉播」操作，屬於互動。

Thetawave.ai

🚗 汽車導航系統建模

類別	職責說明	行為描述	互動關係說明
汽車	執行導航行為	選定目的地	依賴導航系統
導航系統	計算路徑指引	使用最短路徑演算法指引行駛路線	提供導航操作
目的地	導航目標屬性	未明確定義類型	屬性

- 汽車透過導航系統進行導航，形成依賴關係。
- 若導航系統為汽車內建，可進一步衍生**聚合（Aggregation）或組合（Composition）**關係。

🕹️ 遊戲房間與玩家準備狀態

類別	職責說明	行為描述	關聯
遊戲房間	提供準備狀態管理	接收玩家準備訊息	包含玩家（數量不確定）
玩家	發出準備訊號	標示準備	屬於某遊戲房間
遊戲	控制遊戲流程	遊戲自動開始	依賴遊戲房間

- 玩家在遊戲房間內標示準備，是玩家與遊戲房間間的互動行為。
- 遊戲房間收到所有玩家準備後，通知遊戲開始。
- 遊戲與遊戲房間間的關係是依賴，發生於行為之中。

🤖 玩家類別的繼承與多型概念

類別	說明	特殊說明
玩家	抽象類別	基底類別，定義共通職責
AI玩家	玩家子類別	依靠演算法做決策
人類玩家	玩家子類別	從網頁輸入指令做決策

- 「玩家可以選AI或人類玩家」的關係為**is a**（繼承/一般化）。
- 多型（多重繼承）設計可於物件導向設計階段（OOD）詳細規劃，非分析階段重點。

🏫 教室、桌椅與班長模型

類別	描述	行為描述	關聯說明
教室	空間	無直接行為	擁有10組桌椅（聚合關係）
桌椅組	實體	屬性：桌號	歸屬於單一教室
班長	人物	上課前到桌子寫桌號	隸屬於教室

- 桌號為桌椅的屬性，不是獨立實體。

- 教室和桌椅之間為**聚合（Aggregation）**，桌椅受教室控制但可獨立存在。
- 班長與教室為一對一關聯（根據常識判斷）。

Thetawave.ai

📋 董事會與決策投票流程

類別	職責說明	行為描述	關聯說明
董事會	控制決策流程	組織決策會議	包含多位董事（數量不明確）
董事	參與投票	投票（贊成或否決）	可能只參加一個董事會

- 投票後需達到2/3票數通過決議。
- 董事會與董事間的關聯建議賦予有意義的名稱（如「開董事會」）。
- 可用屬性標記董事的投票數或狀態以提升模型完整度。

🌐 HTTP Server 與處理流程模型

類別	職責說明	行為描述	關聯說明
HTTP Server	接收HTTP請求並分派給處理器	接收Request，尋找對應Handler	可擁有多個Handler
Handler	處理HTTP請求	處理Request，回傳Response	屬於一或多個HTTP Server
HTTP Request	請求資料	無直接行為	由HTTP Server與Handler使用
HTTP Response	回應資料	無直接行為	由Handler產生

- HTTP Server與Handler間存在互動，Handler需提供handle操作以處理Request並回傳Response。
- Handler回傳HTTP Response代表Handler創建該實例，形成依賴關係。
- 關係中「創建（Creation）」是一種特殊依賴，標示出Handler對Response的生成責任。

行為導向物件關係總結

概念	說明
行為	實體執行的動作或功能，分析時需明確抓出並分類到相應類別
互動	行為中涉及多個實體協作，需在各實體間開出對應操作以完整描述互動
依賴關係	行為內部才存在的關係，非持久存在，例如「放招打怪物」的依賴
關聯	需求明確定義或基於常識判斷的類別間持久關係，如遊戲與玩家間的含有關係
聚合	一種較強的「擁有」關係，包含整體與部分的生命周期相關性，但部分可獨立存在，如教室與桌椅的關係
組合	更強的整體與部分關係，部分不可獨立存在，如導航系統內建於汽車中
繼承（is a）	類別間代表一般化/特殊化關係，子類別繼承父類別共通屬性與行為

Thetawave.ai

分析階段與設計階段的區分

階段	重點	建議
物件導向分析（OOA）	抓分類（類別）、行為與關係，達到約80%完整度	不用深入考慮設計細節，先完成大框架
物件導向設計（OOD）	細化互動、操作、資料結構與多型設計	利用循序圖等工具，逐步補足遺漏與細節

- 分析階段不需追求100%完整，避免過度思考導致停滯。
- 設計階段可從分析結果中發展出接口（interface）、抽象類別、多行等設計概念。

這些物件導向分析的核心原理與方法相互關聯，形成系統的結構與行為模型，便於後續設計階段的細化與實作。

📂 類別圖中結構與行為的區分與建模

> 類別圖（Class Diagram）用來描述系統中的類別（Class）、它們之間的靜態關係（結構）以及類別所擁有的行為（方法或操作）。理解類別與行為之間的區分，能有效幫助分析複雜系統的脈絡和互動。

---

1. Stereotype（刻板印象）標示：貼標籤的工具

- 在 UML（統一建模語言）中，使用尖角括弧 <<create>> 表示一種特殊的**行為標記**，稱為 **Stereotype（刻板印象）**。
- Stereotype 就像「貼標籤」一樣，用來標示某個操作的特別意義，例如：<<create>> 表示該操作是「建構子」（Constructor），用來建立物件。
- 雖然在初期學習中可以不過度關注 Stereotype，但它有助於清楚表達操作的角色和意圖。

---

2. 領域模型的完整度與依賴關係

概念	說明	例子
依賴（Dependency）	一個類別的行為需要用到另一個類別，這種關係稱為依賴。	Handler 在執行時需要用到 HttpRequest。
領域模型完整度	描述領域模型時，可透過加強類別間的互動細節使模型更完整。	HttpClient 透過 HttpRequest 向 HttpServer 發出請求。

- 依賴關係表示類別之間的互動與呼叫，但不一定是靜態關聯，常用於表示一個類別操作需暫時呼叫另一類別。
- 透過補充行為描述，領域模型更具體，能更準確呈現系統的動態互動。

---

3. Kubernetes (K8s) 的類別圖範例解析

- **Kubernetes 簡介**：

Thetawave.ai

- Kubernetes 簡稱 K8s，因為「K」和「S」間有 8 個字元。
- 是容器化時代的主流技術，管理容器與映像檔。

3.1 類別與關聯描述

類別名稱	角色說明	關聯與數量描述
Kubernetes	系統本身	可**建立多個 Pod**，Pod 與 Kubernetes 是一種組合（Composition）關係，Pod 只能被一個 Kubernetes 系統建立。
Pod	執行單位	每個 Pod 可綁定至少一個映像檔（Image），數量為 1；映像檔則可被 0 到多個 Pod 綁定。
Image（映像檔）	軟體映像	被 Pod 綁定，負責容器執行的內容。
Container（容器）	執行環境	由 Pod 創建並啟動，負責執行映像檔。

- **組合關係（Composition）**：Pod 必須依附於某個 Kubernetes 系統才能存在，故為強組合關係，Pod 無法獨立於 Kubernetes 存在。
- **靜態關係**：Pod 與映像檔的綁定關係反映靜態擁有。

3.2 行為與依賴關係

行為名稱	描述	對應依賴/互動
創建 Pod	Kubernetes 執行「建立 Pod」的行為	Kubernetes 透過 <<create>> Stereotype 標示呼叫 Pod 的建構子。
啟動容器	Pod 被建立後，會啟動容器來執行映像檔	Pod 依賴容器，容器透過「執行映像檔」行為使用 Image，依賴標示為執行。

- **建構子與操作衝突說明**：
  - 「創建」操作本身是一種行為，但類別的存在必須先有物件。
  - 這種矛盾可透過「建構子」的定義（使用 <<create>>）表達，建構子是負責物件初始化的特殊方法。

---

4. 結構與行為的區分重點

類別結構（靜態）	類別行為（動態）
類別（Class）、屬性、關聯（Association）	方法（Operation）、建構子（Constructor）、依賴（Dependency）
反映物件間的靜態擁有與關係	反映物件間的互動、呼叫與依賴
以數字標示關係多寡（多重性）	以 Stereotype 標示特殊行為（如 <<create>>）

- 類別圖中，結構描述了物件如何組成和擁有彼此；行為則描述物件如何彼此呼叫、依賴與互動。
- 行為中的依賴常是暫時性的，體現動態流程。

---

5. 實務應用與建議

Thetawave.ai

- 透過類別圖，能清晰梳理系統中複雜的詞彙與關係，幫助理解技術架構（如 Kubernetes）。
- 使用 Stereotype 明確標示特殊行為，有助於在分析階段區分物件的「存在」與「行為」。
- 掌握組合關係與依賴關係的差異，能更準確描述系統中物件生命週期和互動流程。

Kubernetes 物件關係綜合判斷

Kubernetes 系統、Pod、容器與映像檔之間，以組合與依賴關係緊密結合，從靜態結構到動態行為相互呼應。Pod 必須由 Kubernetes 創建，並進一步建立容器執行映像檔，反映出整體系統運作的層次性和動態互動的本質。

1.16 OOD 階段一_用行為模型來梳理實作.mp4

🎯 物件導向設計（OOD）目標與流程

> **物件導向設計（Object-Oriented Design, OOD）的目標是將在物件導向分析（OOA）階段建立的領域模型，進一步加工成好實作（易於撰寫且正確運作）且好維護（易讀、易改、易擴充）的設計藍圖。**

- **OOA 階段重點**：快速結構化需求，萃取重要知識，建立領域模型（類別圖）。
- **OOD 階段重點**：加工領域模型，聚焦程式架構與實作細節，產出設計藍圖（類別圖升級版）。

OOD 流程分兩大階段

階段	目標	主要工作內容
先求跑（Make it work）	讓模型「動起來」，先達成功能可用	透過**循序圖（Sequence Diagram）**梳理流程與類別間互動，修改領域模型以補足漏失的操作
再求好（Make it great）	追求優良設計與易維護性	依序進行封裝、解耦、萃取，學習設計模式、設計原則（如 SOLID、乾淨架構）

🏗️ 結構模型（Structural Model）與行為模型（Behavioral Model）

結構模型（類別圖）

- 反映需求中重要的概念（類別）及其關係。
- 展示類別的屬性、方法及驗證規則（例如：玩家數量固定為 4，人數、手牌數量限制等）。
- 缺點：無法清楚呈現類別間的時間順序與互動流程。

行為模型

- 補足結構模型不足，描繪類別間的互動流程。
- 早期行為模型示意：在類別圖上強行加入時間維度，標示呼叫順序與互動。
- 缺點：難畫、難理解且會破壞類別圖結構。

Thetawave.ai

🗂️ 循序圖（Sequence Diagram）介紹與使用

循序圖目的

- 將類別間的互動以時間軸方式展現，幫助快速找出需求流程中的漏失操作。
- 使領域模型能更完整、可實作，避免程式撰寫時遇到明顯的職責分配不全問題。

循序圖的基本元素

元素名稱	表示方式	功能說明
物件（Object）	:類別名稱 或 物件名稱:類別名稱	表示參與互動的實體物件，每個物件佔據一條垂直生命線
生命線（Lifeline）	物件下方的虛線	表示物件存在期間，從上到下代表時間流逝
工作條（Activation Bar）	物件生命線上的長方形	表示物件正在執行工作（方法呼叫期間）
呼叫箭頭（實線）	由呼叫者指向被呼叫者	表示方法呼叫，呼叫者也稱為**Client（客戶端）**
回傳箭頭（虛線）	從被呼叫者指向呼叫者	表示方法執行完畢，控制權回傳給呼叫者
變數與回傳值	呼叫後可將回傳值賦予變數	用於後續方法呼叫的參數傳遞
控制流程運算子	以大框框標示	表示流程控制結構，如 loop（迴圈）、if-else（條件分支）

循序圖操作流程（以撲克牌遊戲為例）

1. **宣告物件**：遊戲（Game）、牌堆（Deck）、玩家（Player）
2. **開始遊戲**：遊戲物件呼叫開始遊戲操作，並傳入玩家清單作為參數。
3. **洗牌**：遊戲呼叫牌堆的洗牌方法，牌堆工作完後回傳控制權。
4. **發牌**：
   - 遊戲呼叫牌堆抽牌方法，取得新牌並存入變數。
   - 遊戲呼叫玩家的加入手牌方法，將新牌加入玩家手牌。
   - 以上動作透過迴圈（loop）結構表示，模擬 52 張牌輪流發給 4 位玩家。
5. **出牌階段**：
   - 模擬 13 個回合，每回合玩家輪流出牌。
   - 仍使用迴圈結構控制流程。
6. **結算勝利者**：
   - 遊戲物件呼叫自身的結算方法，印出分數最高的玩家。
7. **控制權回傳**：最終流程結束後，將控制權回傳給遊戲的 Client。

重要觀念：Client

> **Client（客戶端）**是指呼叫某個操作（方法）的物件，了解 Client 有助於分析類別間的職責分配及互動。

💡 透過循序圖發現「漏開操作」

- 例如在發牌流程中，僅呼叫牌堆抽牌但未呼叫玩家加入手牌的操作即為漏失。
- 透過循序圖模擬流程，能在程式撰寫前揭露這類漏失，避免實作卡關。

Thetawave.ai

🔄 循序圖與類別圖的迭代關係

- 循序圖幫助理清互動流程，發現新增操作需求。
- 類別圖補充新增操作與職責分配。
- 兩者交互迭代，逐步完善設計藍圖。

🌟 水球流 OOAD 特點

- **輕量且實用**：保留三大核心模型——類別圖（結構模型）、循序圖（行為模型）、狀態基圖（尚未教授）。
- **強調快速迭代與實作**：先求跑，再求好，避免完美主義導致拖延。
- **連續技武功**：透過模型間相互配合，形成有條理且流暢的設計思考流程。

下一步：OOD 維護性三大基石

- **封裝（Encapsulation）**：將資料與操作包裝在類別內部，避免外部直接存取。
- **解耦（Decoupling）**：降低類別間的依賴度，提升系統彈性。
- **萃取（Extraction）**：從複雜中抽取共通職責，形成清晰的模組。

這三大基石是後續學習設計模式與撰寫高品質程式碼的基礎。

物件導向設計（OOD）流程分析

物件導向設計透過結構模型（類別圖）與行為模型（循序圖）的互動，建立明確的類別職責與流程控制，進而產出既能快速實作又便於維護的設計藍圖。循序圖的時間維度與控制流程運算子有效補充了類別圖的靜態資訊，使得系統行為與類別互動清晰明確。藉由反覆調整循序圖與類別圖，設計者能不斷完善模型，避免遺漏關鍵操作，確保程式碼可順利且正確地執行。這種結構與行為模型的同步迭代，為後續封裝、解耦及萃取等設計優化奠定基礎，促使系統架構更加穩健且易於擴充。

1.17 用 Astah 循序圖來讓[象棋]領域模型動起來.mp4

♟️ 象棋案例的結構與行為整合

> 本案例聚焦於如何從物件導向分析（Object-Oriented Analysis, OOA）轉向物件導向設計（Object-Oriented Design, OOD）階段，透過UML（統一建模語言）類別圖與循序圖（Sequence Diagram）來呈現象棋遊戲的主流程及物件間互動。

---

1. 象棋遊戲的主要類別與結構

類別名稱	角色與職責說明	重要細節
象棋遊戲	代表整個棋盤遊戲，管理棋子擺放、開始遊戲行為	負責遊戲流程的啟動
棋盤（棋子容器）	管理並布置棋子	棋子數量為1到32枚

Thetawave.ai

棋子	各種棋種的實例，有不同的移動規則	移動前需進行合法性驗證
玩家	區分AI玩家與真人玩家	AI玩家為隨機決策，真人玩家透過命令列介面（CLI）輸入指令

- **玩家類別**是新增的實體，負責做出下棋決策。
- 玩家類別可分成兩種：
  - **AI玩家**：透過隨機方式進行決策。
  - **真人玩家**：透過終端機的指令介面（Command Line Interface, CLI）輸入決策。

---

2. 循序圖的建立流程與主流程設計

2.1 建立循序圖的步驟

- 在 UML 編輯器中，對目標大Package點擊右鍵，選擇「Create a Diagram」→「Sequence Diagram」。
- 在循序圖中，依序建立三個主要物件：
  - 象棋遊戲
  - 棋子（視為棋子類別的多個實例）
  - 玩家

2.2 主流程的關鍵互動

流程步驟	行為描述	操作與回傳
遊戲開始	呼叫「開始遊戲」操作，遊戲流程啟動	startGame()：無回傳值（Void）
棋子擺放	遊戲開始時擺放32枚棋子，遞回呼叫強調此行為	setupPieces()（象棋遊戲內部呼叫）
玩家輪流下棋	玩家做出下棋決策，呼叫玩家物件的決策產生操作	makeMoveDecision()：回傳一個決策物件（Move）
取得棋子	根據決策中的來源位置(Row, Column)取得該棋子	getPiece(row, column)：回傳棋子或無（Nullable）
棋子移動	請棋子根據決策的目的地坐標移動	move(toRow, toColumn)：無回傳值
決策合法性檢查	透過Guard條件判斷是否有找到棋子，非法決策則不執行移動	Guard：[找到棋子]
遊戲結束判斷	判斷有人被將死則結束遊戲，呼叫印出勝利者操作	printWinner()：無回傳值

---

3. 重要UML元素及設計細節說明

3.1 操作宣告與一致性

- 在類別圖中，所有操作（Operation）需事先宣告，否則在循序圖中會顯示為「Unspecified」。
- 透過編輯器的「New」按鈕快速新增操作，並指定回傳型別與參數。
- 例如，startGame()宣告為無回傳值，makeMoveDecision()回傳決策物件，getPiece()需帶入Row、Column兩個參數。

3.2 決策物件（Move）的結構

- 決策物件封裝了玩家下棋的資訊，包含四個關鍵欄位：

Thetawave.ai

欄位名稱	代表意義
Row	棋子目前所在列
Column	棋子目前所在欄
TwoRow	棋子目標位置的列
TwoColumn	棋子目標位置的欄

- 決策物件用來傳遞玩家的移動指令，讓遊戲系統理解要移動哪個棋子到哪裡。

3.3 依賴關係與建立（Create Stereotype）

- 玩家透過makeMoveDecision()建立一個決策物件（Move），此為「建立的依賴關係」（Create Dependency）。
- 在UML中，可使用標記（Stereotype）<<create>>來表示物件的建立關係。

3.4 Guard（前提條件）

- Guard是UML中用來表達條件判斷的符號，寫在中括號內。
- 例如：[找到棋子]表示只有在該條件成立時，才會執行後續的移動操作。
- Guard可限制操作呼叫的執行時機，避免非法操作發生。

3.5 迴圈（Loop）結構

- 使用「Combine Fragment」的Loop運算子來表示重複執行的流程。
- 例如：遊戲進行中，玩家輪流下棋直到有人被將死。
- Loop同樣可帶Guard條件，如「直到有人被將死之前持續執行」。

---

4. 物件導向設計的進階思考

設計階段	說明
封裝（Encapsulation）	將資料與操作包裝在類別內，保護內部狀態
私有操作（Private）	非公開給外部使用的操作，僅供類別內部呼叫
多形（Polymorphism）	玩家類別可定義為抽象類別，具備抽象操作，具體子類實現不同決策策略
抽象類別（Abstract Class）	在精煉階段引入抽象類別，定義共通介面，提升系統擴展性

- 初期分析階段不需使用抽象類別，直到設計精煉階段再導入。
- 多形允許AI玩家和真人玩家以同一介面操作，但行為細節由子類別實現。

---

5. UML 設計工具輔助功能

功能名稱	功能說明
Operation 新增	快速在類別中新增操作並設定回傳型別與參數
Create Object	直接點兩下可建立物件實例於畫布中
Argument 設定	呼叫操作時可設定參數的具體值（例如 Move.Row, Move.Column）
Combine Fragment	建立複合結構（如迴圈、條件分支）
Guard 設定	為操作呼叫加上條件判斷
Gap Expander	一鍵調整模型元素間距，保持畫面整齊
Gap Remover	一鍵縮小模型元素間距

Thetawave.ai

6. 象棋遊戲主流程的行為示範（循序圖核心流程）

> 透過循序圖描述物件間的互動與訊息傳遞，清晰展示遊戲從開始到遊戲結束的流程：

1. Client 呼叫象棋遊戲物件的 `startGame()`。
2. 象棋遊戲呼叫自身 `setupPieces()`，擺放棋子。
3. 遊戲進入迴圈，玩家依序呼叫 `makeMoveDecision()`。
4. 象棋遊戲根據決策，呼叫 `getPiece(row, column)`取得棋子。
5. 若找到棋子（Guard成立），棋子執行 `move(toRow, toColumn)`。
6. 判斷是否有人將死，若否，回到玩家輪流下棋步驟。
7. 若有人將死，呼叫 `printWinner()`，結束遊戲。

---

7. 設計思考與實作建議

- **結構與行為的同步**：先在類別圖宣告操作，再在循序圖中設計流程，保持模型一致。
- **細節漸進精煉**：先完成基本流程，再逐步加入條件判斷、資料結構優化與多形設計。
- **循序圖為思考輔助工具**：循序圖重點是理清物件互動，不需過度追求完美排版，思路清晰最重要。
- **私有化操作**：開始遊戲（`startGame()`）為公開操作，其他輔助操作可設為私有，保護封裝性。
- **抽象化玩家行為**：可將玩家定義為抽象類別，具體玩家（AI、真人）分別實作決策方法。

象棋遊戲的物件導向設計將遊戲流程拆解為多個明確的操作與物件協作，透過UML類別圖與循序圖的配合，能夠有效地將抽象需求轉化為具體可實作的系統結構。設計過程中，條件判斷與迴圈控制使遊戲流程更加嚴謹，封裝與多形的引入提升系統的可擴展性與維護性。

1.18 OOP 技法一_依賴.mp4

👨‍💻 物件導向程式設計（OOP）基礎與依賴關係實作

> **物件導向程式設計（Object-Oriented Programming, OOP）** 主要是將設計藍圖轉換成程式碼，強調類別（class）、屬性（attribute）、方法（method）以及物件間的關係實作。
> 本單元重點是掌握如何依設計圖正確實作類別與其約束（constraint），特別是「依賴關係（dependency）」的程式實作。

---

1. OOP 實作的核心觀念與準備

- **程式語言不限**：只要支援物件導向的語言皆可（如 Java、Python、C++、C#、TypeScript 等），重點是學會設計轉譯思維，不必拘泥於語法細節。
- **設計藍圖（Class Diagram）為依據**：將藍圖中的類別、屬性、方法一一對應實作。
- **重點不是語法，而是實作規則**：不熟 Java 可用自己熟悉的語言，但需了解該語言如何定義類別、屬性與方法。

Thetawave.ai

2. 範例實作：英雄（Hero）與等級表（Level Sheet）

2.1 類別與屬性設計

類別名稱	屬性（初始值）	方法（行為）	備註
Hero	level: 1	gainExperience(exp)	以經驗值查等級表更新等級
	totalExperience: 0		屬性必須守護約束
	hp: 100		血量（health points）
LevelSheet	無屬性	queryLevel(totalExperience)	根據總經驗值回傳對應等級

2.2 屬性約束（Constraint）守護策略

- **統一設定點（Unified Setter）**
  所有屬性的更新必須透過一個**唯一的 setter 方法**，才能在該方法中集中驗證屬性的值是否符合約束。
- **私有化 setter（private setter）**
  setter 不對外公開，避免外部直接修改，僅類別內部可使用，外部改變屬性需透過有業務邏輯的方法（例如 gainExperience）。
- **屬性約束範例**：

屬性	約束條件	實作策略
totalExperience	必須 ≥ 0	setter 中驗證，不符拋出例外
level	必須 > 0，且與等級表對應	更新前查表，透過 setter 驗證
hp	必須 ≥ 0	setter 中驗證，不符拋出例外

- **跨屬性不變條件**
  例如 level 必須與 totalExperience 在等級表中對應正確，須查表後才能更新。

---

3. 行為方法（Behavior Method）設計：gainExperience

3.1 方法前提條件驗證

- 傳入的經驗值 exp 必須 **大於等於 0**，否則立即拋出例外（exception）。
- 這種方法內的前提條件驗證是以 if-else 判斷實現。

3.2 行為流程

1. 將傳入的經驗值加到 totalExperience 屬性（透過 setter 以確保約束）。
2. 呼叫 LevelSheet 的 queryLevel 方法，傳入新的 totalExperience，取得對應的新等級。
3. 以 setter 更新 level，同時確認不違反約束條件。

---

4. 等級表（LevelSheet）查表邏輯

- **等級計算規則**：

$$\text{等級} = \left\lfloor \frac{\text{totalExperience}}{1000} \right\rfloor + 1$$

，例如：

Thetawave.ai

- 經驗值 0 → 等級 1
- 經驗值 1000 → 等級 2

- queryLevel 回傳此計算結果作為英雄的當前等級。

---

5. 依賴關係（Dependency）的程式實作

- **定義**：依賴關係指某個類別的方法在執行時，需要「暫時使用」另一個類別的物件。
- **實作方式**：將依賴物件（此例為 LevelSheet）「作為參數」傳入使用此物件的方法（如 gainExperience）。
- **特點**：依賴是最簡單的動態關係，不需在類別中永久保存參考，只是偶爾使用。

---

6. 類別設計與屬性、方法的實作流程總結

步驟	內容說明
1. 宣告類別	根據設計藍圖宣告類別、屬性與方法，設定初始值
2. 守護屬性約束	透過私有 setter 建立統一的屬性更新點，並於 setter 中執行數值驗證與不變條件確認
3. 行為前提驗證	在方法一開始檢查參數是否符合前提條件，不符則拋出例外
4. 實作依賴關係	將依賴物件作為參數傳入方法，建立動態依賴，不需永久保存引用

---

7. 測試程式設計

- 建立 Hero 與 LevelSheet 物件。
- 透過 gainExperience 測試不同經驗值輸入（0、100、800、900、-200）。
- 預期結果：
  - 經驗值 ≥ 0 時，英雄等級依經驗值升級（例如 0 維持等級 1，1000 升等級 2）。
  - 經驗值 < 0 時，拋出例外錯誤訊息（"The gain experience must be greater than or equal zero"）。

---

8. 重要設計原則與實作技巧

主題	重點說明
封裝（Encapsulation）	利用私有屬性與方法限制外部直接修改，保護物件狀態的正確性
統一設定點	所有屬性更新必須透過唯一 setter，強制執行約束條件檢查
例外處理	不符約束條件時丟出例外，確保資料不被破壞
跨屬性不變條件	例如 level 與 totalExperience 必須符合等級表規則，需在更新時即查表確保一致性
動態依賴實作	依賴物件透過參數注入，建立方法內短暫使用的關係

Thetawave.ai

9. 物件導向設計與實作的意義

物件導向設計藉由嚴謹守護屬性約束與行為前提，確保物件狀態永遠保持一致且合理。統一的設定點提供了集中管理狀態變更的機制，避免分散修改導致錯誤。動態依賴則讓物件間的關係更靈活，降低耦合度，提高維護性與可測試性。這些實作細節彼此緊密相關，形成一個嚴謹且高品質的程式碼架構。

1.19 OOP 技法二_關聯.mp4

🐾 英雄與寵物的一對一雙向關聯

需求背景與物件導向分析

- 英雄（Hero）可以擁有一隻寵物（Pet）
- 每隻寵物最多只有一個主人（Owner）
- 寵物吃任意水果時，主人英雄的血量會增加 10 點（10d）
- 英雄與寵物之間的關聯為 **0 到 1 個**（nullable）

類別設計與屬性宣告

類別	屬性名稱	說明	允許值
Hero	pet	持有一隻寵物，nullable	null 或 Pet
Pet	owner	寵物的主人，nullable	null 或 Hero

- **nullable**：代表該屬性可以為 null，表示沒有持有或沒有主人
- Hero 中在建構時未必有寵物，因此 pet 屬性允許為 null
- Pet 的 owner 也允許為 null，表示可能沒有主人

實作重點

- Hero 類別中透過屬性直接持有 Pet 實例，實現「has-a」關係
- 實作單向關聯時，只需在 Hero 端宣告 pet 屬性，提供 setter/getter 即可
- 實作雙向關聯時，必須在 Hero 的 setPet() 方法中，同時呼叫 Pet 的 setOwner()，以維持雙方關聯一致性
- Pet 的 eat 操作中必須確認 owner 是否為 null，若有主人則增加主人血量 10d
- 為避免 null pointer，eat 方法中需判斷 owner 是否為 null 再執行增血

養寵物與棄養的邏輯

- **養寵物（setPet）**：
  - 若已有寵物，先呼叫舊寵物的 setOwner(null) 將主人清除（棄養）
  - 再將新寵物設為 hero 的 pet
  - 呼叫新寵物的 setOwner(this) 設定主人
- **棄養寵物（removePet）**：
  - 呼叫現有寵物的 setOwner(null)
  - 將 hero 的 pet 設為 null
- 以此方式確保雙向關聯在任何時候均保持一致

範例測試流程

1. 建立 hero，初始血量 100
2. 建立寵物（例如貓）
3. hero 設定寵物（setPet）

4. 寵物連續吃 5 次不同水果，hero 血量每次增加 10d
5. hero 棄養寵物（removePet）
6. 寵物再吃水果時 hero 血量不再增加

Thetawave.ai

🏰 英雄與公會的多對多雙向關聯

需求背景

- 每位英雄（Hero）可加入多個公會（Guild），0 到多個
- 每個公會有名稱（name）
- 公會成員數量限制為 1 到 10 人，不能成立沒有成員的公會
- 公會提供加入（join）與離開（leave）成員的操作
- 加入時不得重複加入
- 離開時必須是該公會成員且離開後成員數不可少於 1 人

類別屬性與資料結構

類別	屬性名稱	資料型態	說明
Guild	name	String	公會名稱
Guild	members	List 或 Set<Hero>	公會成員集合
Hero	guilds	List 或 Set<Guild>	英雄參與的公會集合

- 建議使用 **Set** 代替 List 以避免重複元素
- Guild 成員數量限制 1 到 10，必須在建構子強制要求初始成員數量符合條件

雙向關聯的實作原則

- Guild 為「擁有方」（Owner），負責建立與維護雙向關聯
- Guild 的 join(member) 會：
  - 驗證成員數未達 10，且 member 尚未加入
  - 將 member 加入 members 集合
  - 呼叫 member.addGuild(this) 使 Hero 被動加入公會列表
- Guild 的 leave(member) 會：
  - 驗證成員數量大於 1，且 member 是成員
  - 將 member 從 members 集合移除
  - 呼叫 member.removeGuild(this) 使 Hero 被動移除公會列表

Guild 建構子

- 必須接收至少一位 Hero 作為初始成員，確保公會成立時成員數量合法
- 建構子中驗證 members 集合大小 $\in [1, 10]$
- 建構時呼叫所有初始成員的 addGuild(this) 以建立雙向關聯

Hero 類別對應方法

- addGuild(Guild) 將 Guild 加入 hero 的 guilds 集合
- removeGuild(Guild) 從 hero 的 guilds 集合移除
- 提供不可修改版本的 guilds 集合 getter，避免外部破壞關聯

公會成員集合的保護

- 對外暴露的成員集合使用不可變封裝（例如 Java 的 Collections.unmodifiableList 或 unmodifiableSet）
- 防止外部客戶端直接修改成員集合，破壞基數限制與關聯一致性

Thetawave.ai

成員加入與離開的前置檢查

行為	檢查條件	不符合條件時反應
加入公會(join)	1. 成員數未滿 10 2. 成員尚未加入該公會	丟出例外，拒絕加入
離開公會(leave)	1. 成員數大於 1 2. 成員已是該公會成員	丟出例外，拒絕離開

測試案例示意

- 建立三位英雄 A、B、C
- 建立公會，初始成員為 A
- B、C 相繼加入公會，成員數增加
- 嘗試讓 A 重複加入公會，預期丟出例外
- 讓 A、B 離開公會，剩 C 一人
- 讓 C 嘗試離開公會，預期丟出例外（成員數不可低於 1）
- 讓非成員 A 嘗試離開，預期丟出例外（非成員無法離開）

🔄 單向與雙向關聯的差異與轉換

關聯類型	描述	實作特點
單向關聯	一方持有另一方的參考，另一方不持有反向參考	只需在持有方類別定義屬性與 setter/getter
雙向關聯	雙方互相持有對方參考	需在一方（擁有方）負責同步維護雙向關聯的一致性

- 若將雙向關聯改為單向，可刪除不需要的反向屬性與方法，降低耦合
- 單向關聯適用於只有一方需要存取對方的情境
- 雙向關聯可確保雙方狀態同步，但實作較複雜，需小心維護關聯一致性

📋 物件導向關聯實作規則總結

關聯類型	持有方	持有屬性類型	典型實作重點
一對一單向	Hero	單一 nullable 寵物（Pet）	只需在 Hero 中宣告 pet 屬性，setter/getter
一對一雙向	Hero	單一 nullable 寵物（Pet）	Hero 的 setPet 同時呼叫 Pet 的 setOwner
一對多雙向	N/A（未提及）	多個對象集合	集合中新增/刪除需同步更新雙方關聯
多對多雙向	Guild	Set<Hero> 與 Set<Guild>	Guild join/leave 負責雙向同步，Hero 被動接受

- 關聯的基數（cardinality）影響屬性型態（單一或集合）及 nullable 設定
- 雙向關聯需在擁有方建立並維護雙向一致性
- 對集合的公開需包裝不可變結構，避免外部直接修改破壞關聯條件

- 關聯建立或移除時須加入前置條件檢查，避免違反業務規則

Thetawave.ai

雙向關聯的維護與實踐意義

雙向關聯透過雙方互相持有對方參考並在業務方法中同步更新，形成一個閉環的物件網絡，有助於維持物件之間的狀態一致性與業務邏輯的完整性。擁有方負責統籌管理關聯的建立與拆除，並執行嚴格的前置條件驗證，使關聯狀態始終符合業務規範。這樣的設計在實務中強化系統的健壯性，防止錯誤狀態蔓延，保障整體物件模型的正確性與可維護性。

🐾 單向與雙向關聯的基礎概念

> **關聯（association）**指的是物件間「有」或「沒有」的靜態關係，在物件導向程式設計（OOP）中非常重要。

- **單向關聯**：只有一方持有另一方的參考，例如 Hero 持有 Pet，但 Pet 不持有 Hero。
- **雙向關聯**：雙方互相持有對方的參考，例如 Hero 持有 Pet，Pet 也持有 Hero。

🎯 單向關聯的實作細節

- 如果 Hero 可有可無寵物，則 Pet 屬性必須標記為 **Notable Type（可選型別）**，表示可能為空值（null）。
- 為避免直接存取空值導致的 Null Pointer Exception（空指標異常），必須在使用前判斷是否為空。
- 不同程式語言標記 Notable Type 的方式：

程式語言	Notable Type 標記方式
Java	使用特殊註解 `@Nullable`
Kotlin	型別後加問號，如 `Pet?`
TypeScript	使用聯合型別，如 `Pet | null`
Python	使用型別提示 Optional，預設為 `None`

- 傳入寵物（Pet）的方式：
  - 如果寬鬆：建構子不強制傳入 Pet，後續可用 Setter 注入。
  - 如果嚴格：建構子直接傳入 Pet。

🔄 雙向關聯的設計與實作

雙向關聯的核心要點

- 雙方各自持有對方作為屬性，互相知道彼此。
- 需要明確區分「擁有方（Owner）」與「被擁有方（Passive）」。
- 擁有方負責管理雙向關聯的建立與維護。

以 Hero 和 Pet 為例

角色	職責說明
Hero	擁有方，負責設定 Pet，呼叫 Pet 的 `setOwner()` 設定自己的引用。

Pet	被動方，接受並儲存 Hero 的參考。

Thetawave.ai

雙向關聯建立步驟

1. **建構子注入**：Hero 的建構子帶入 Pet，建立初始關係。
2. **Setter 方法**：Hero 的 `setPet()` 或 `addPet()` 等方法負責設定 Pet，並呼叫 Pet 的 `setOwner()`。
3. **Pet 接收**：Pet 接收 Hero 的參考，完成雙向鏈結。

⚖️ 0~1 與 0~多數雙向關聯的業務邏輯挑戰

0~1 雙向關聯（Hero 0~1 只寵物，Pet 0~1 個主人）

- 需要嚴謹管理狀態變更，避免同時持有多個寵物或主人。
- 可能遇到的問題：

問題情境	解決方案
Hero 已有寵物，嘗試設定新寵物	1. 拋出例外，禁止操作 2. 允許覆蓋，先放棄原寵物
Pet 已有主人，新增另一個主人	同上

- 程式邏輯必須嚴格守護關聯條件，避免不一致狀態產生。

0~多 雙向關聯（Hero 擁有多隻寵物，Pet 有多個主人）

- Hero 使用集合（Set、List、Map）存放多個 Pet。
- Pet 也使用集合存放多個 Hero（Owner）。
- 雙向關聯維護更複雜，必須同步更新雙方集合。

📊 Hero 與 Pet 雙向關聯的資料結構與方法設計

角色類別	屬性類型	關聯範圍	方法示例	說明
Hero	`Set<Pet>`	0~多隻寵物	`addPet(Pet p)`, `removePet(Pet p)`	增加/移除寵物時同步呼叫 Pet 的 `addOwner()` 或 `removeOwner()`
Pet	`Set<Hero>`	0~多個主人	`addOwner(Hero h)`, `removeOwner(Hero h)`	被動接受或移除主人參考

雙向關聯同步示意

```
Hero.addPet(p):
    pets.add(p)
    p.addOwner(this)

Pet.addOwner(h):
    owners.add(h)
```

- `removePet` 和 `removeOwner` 同理，雙方同步移除。

Thetawave.ai

🛡️ 關聯條件的守護與例外處理

- **守護原則**：程式碼中必須嚴格檢查關聯條件，防止異常狀態。
- **常見守護機制**：

情況	措施說明
重複加入同一對象	利用 Set 自動去重或手動檢查
設定新寵物時已有舊寵物存在	拋出例外或先移除舊寵物
移除不存在的關聯	可選擇忽略或拋出警告

- 需求端與程式設計者需溝通清楚「允許行為」與「禁止行為」，確保邏輯一致。

🔄 建構子注入與後續關聯管理策略

- **建構子注入**：在創建 Hero 時，可直接注入 0、1 或多隻 Pet。
- **後續管理**：使用 addPet、removePet 方法維護狀態。
- 建構子內實作細節：

```
for each pet in pets:
    addPet(pet)  # 利用方法來建立雙向關聯，確保一致性
```

- 透過這種方式，雙向關聯在物件建立時即完整且正確。

🧩 領域模型（Domain Model）與程式碼設計心態

- **領域模型**的視覺化與結構化，幫助開發者更沉著應對複雜業務邏輯。
- **分割擊破策略**：將複雜問題拆解為多個小問題逐一落實，降低錯誤率。
- 透過領域模型的清晰表達，程式碼更容易維護、讀懂與擴充。

物件關聯的實踐要點

關聯類型	基本意義	屬性擁有方	代表語義
單向關聯	一方持有另一方參考	持有方	Hero 有 Pet
雙向關聯	雙方互相持有參考	由擁有方主導維護	Hero 與 Pet 互為對方持有方

- 關聯屬於**組合（Composition）**的一種，展現物件間的靜態結構。
- 關聯的正確實作是程式碼基礎穩固與可維護的關鍵。

雙向關聯的設計與實作要求掌握擁有方與被擁有方的明確角色定位，並透過適宜的資料結構與方法同步管理雙方參考，確保關聯條件的嚴格守護以避免狀態不一致。領域模型的視覺化使複雜業務邏輯逐步拆解實作，提升程式碼的可讀性與穩定性。物件間的關聯作為組合關係的核心，決定了系統架構的基礎維護性與擴展性。

Thetawave.ai

1.20 OOP 技法三_關聯類別.mp4

👨‍🏫 一對一雙向關聯與關聯類別的基本需求

> 「每個學生每學期只能修一門特別課（lecture），每堂特別課也只能有一位學生。」

- 這是**一對一雙向關聯**的典型例子。
- 關聯雙方：Student（學生）與 Lecture（特別課）。
- 關聯類別（Association Class）出現的原因：學生在課堂中會有一個 **grade（成績）** 屬性，成績是關聯的附加資訊，不能直接放在學生或課程類別中，必須用中介類別來表示。

🏗️ 關聯類別（Association Class）的結構與實作原理

- **關聯類別**是用來表示兩個類別之間的關聯，且該關聯擁有自己的屬性與行為。
- 它像是一個「中間人」，雙方不直接持有彼此，而是持有這個關聯類別的實例。
- 關聯類別會持有雙方的參考（Student、Lecture）。
- 例如：LectureAttendance 類別，代表某位學生在某門課的參與與成績。

關聯類別的主要組成：

元素	說明
參考雙方	透過建構子接收學生與課程物件
附加屬性	例如成績 grade，初始為 null（可空值）
行為	例如 receiveGrade 方法用來設定成績

🎯 重要概念：成績屬性的類型處理與驗證

- **成績** grade 是一個整數，範圍限定在 0 到 100。
- Java 等語言的原始型別（primitive type）不可直接設為 nullable，因此需包裝（boxing）成可空物件（如 Integer）。
- 驗證成績範圍時，使用可重複利用的 validationUtils，允許彈性設定上下限，確保輸入合理。

🔄 雙向關聯的建立與解除

報名（sign up）

- 在 Lecture 類別中實作 signUp(Student s)：
  - 檢查該學生是否已報名其他特別課（因為只能修一門）。
  - 檢查該課程是否已有學生報名（因為每門只收一位）。
  - 通過檢查後，建立 LectureAttendance 物件，並同時設定學生與課程的參考，建立雙方的雙向關聯。

取消報名（sign off）

- 必須先確認學生已報名該課。

- 解除 LectureAttendance 物件的雙向參考（設定為 null）。
- 若不符合條件（未報名或未在該課報名），拋出例外。

Thetawave.ai

🛠️ 程式碼實作細節與設計考量

類別	屬性與方法
Student	name、nullable lectureAttendance (或多筆清單) 提供 getter、被動接受新增或移除關聯的方法
Lecture	name、nullable lectureAttendance (或多筆清單) signUp(Student)、signOff(Student) 方法
LectureAttendance	參考 Student、Lecture grade（nullable）、receiveGrade(int) 驗證並設定成績

- **nullable** 用於可不存在的關聯物件，避免 NullPointerException。
- 使用 setter/getter 管理關聯物件，並在變動時同步更新雙方狀態。

➕ 從一對一到多對多的關聯類別擴展

關聯類型	學生端持有數量	課程端持有數量	程式碼變動重點
一對一	0 或 1（nullable）	0 或 1（nullable）	持有單一 LectureAttendance 實例，getter/setter調整nullable
一對多	多（List）	0 或 1（nullable）	學生端改持有多筆 LectureAttendance，新增 add、remove 方法
多對多	多（List）	多（List）	兩端皆改持有多筆 LectureAttendance，signUp/signOff 增加查詢與管理邏輯

- 多筆關聯改用集合（List）儲存，並封裝為不可修改的集合回傳。
- 增加新增與移除關聯的方法以維護一致性。
- 查詢時需要遍歷集合比對學生或課程身分以防止重複報名。

🔍 查詢與例外處理邏輯示意

操作	查詢條件	例外情況
報名	學生是否已報名該課程 (查詢 LectureAttendance list)	學生已報名該課程或已修其他課，拋出例外
取消報名	是否存在該學生與課程的 LectureAttendance 物件	找不到該關聯物件，表示學生未報名該課程，拋出例外

🧪 測試案例與執行情況

- 建立學生 Johnny 與特別課軟體設計模式精通之旅。
- Johnny 報名該課成功，分數設為 60 分。
- 重複報名時拋出例外。

Thetawave.ai

- 取消報名後，重複取消拋出例外。
- 多課報名範例：學生同時報名多門課程，成績分別設定，並成功取消報名。
- 多學生報名同一課程的多對多案例，確保雙方關聯正確管理。

代碼設計的關鍵要點

- 關聯類別必須在建構時即持有雙方物件，確保關聯不會中斷。
- 雙向關聯必須同步更新雙方參考，防止資料不一致。
- 以行為（sign up、sign off、receive grade）封裝關聯操作，限制非法狀態。
- 利用例外處理防止重複報名與非法取消。
- 使用不可變集合（unmodifiable list）回傳關聯，提升封裝與維護性。
- 驗證邏輯抽離為工具類別，方便重複使用與維護。

透過對關聯類別的中介設計，雙向關聯能夠正確管理附加資訊與狀態，並且靈活調整關聯的多重性，使得物件關係在程式中更具結構性與一致性。